



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

السنة الرابعة

قسم الذكاء الاصطناعي

مشروع البرمجة التفرعية

خطو عنوان المشروع

سليم ابراهيم بركات

اية محمد خير الخطيب

سيدرا عبدالله

اميرة ايمن كلش

نائل مامون مهنا

المحمود

بإشراف: حذيفة عقيل

#1 تقرير مشكلة حالة السباق (Race Condition) في الواجهة البرمجية لسلة التسوق (Shopping Cart API):

يشرح هذا المستند مشكلة برمجية حرجة تُعرف بـ **حالة السباق (Race Condition)** في نظام سلة تسوق مبني باستخدام Node.js و MongoDB، وكيفية حدوثها عند تدفق طلبات متزامنة بكثافة عالية في نفس اللحظة، وكيف تم حلها جذرياً باستخدام **العمليات الذرية لقاعدة البيانات (Atomic Database Operations)**.

1. المشكلة: حالة السباق في تحديثات سلة التسوق

عندما تحاول عدة طلبات متزامنة تحديث نفس العنصر داخل سلة التسوق في نفس الوقت، يعتمد النظام على نمط تقليدي يُعرف باسم **(Read-Modify-Write) أي "القراءة ثم التعديل ثم الكتابة"**:

1. **قراءة (Read):** جلب بيانات سلة التسوق الحالية من قاعدة البيانات.
 2. **تعديل (Modify):** تعديل الكمية وزيادتها داخل ذاكرة الخادم (In-memory).
 3. **حفظ (Write):** حفظ المستند كاملاً وإعادته إلى قاعدة البيانات.
- هذا النمط يتسبب في حدوث **حالة سباق (Race Condition)** عند معالجة طلبات متعددة ومتزامنة.

العواقب المترتبة على المشكلة

- تقوم بعض الطلبات بالكتابة فوق الطلبات الأخرى وتلغيها (Overwriting).
- فقدان وضياح تحديثات الكميات المطلوبة (Lost Updates).
- تصبح الحالة النهائية لسلة التسوق غير متناسقة ولا تعكس الطلبات الحقيقية للمستخدم.

- مثال توضيحي تحت ضغط الطلبات المتزامنة

- **الكمية المتوقعة (Expected Quantity): 50**
- **الكمية الفعلية المسجلة (Actual Quantity): 2**
- **التحديثات المفقودة (Lost Updates): 48**

هذا يعني أن 48 تحديثاً قد ضاعت تماماً بسبب عمليات الكتابة المتزامنة الخاطئة!
2. الكود البرمجي القديم (قبل التعديل - الكود المسبب للمشكلة)

```
exports.addProductToCart = asyncHandler(async (req, res, next) => {
  const { productId, color } = req.body;
  const product = await Product.findById(productId);
  let cart = await Cart.findOne({ user: req.user._id });
  if (!cart) {
    cart = await Cart.create({
      user: req.user._id,
      cartItems: [
        {
          product: productId,
          color,
          price: product.price,
          quantity: 1,
        },
      ],
    });
  }
});
```

مشروع البرمجة التفرعية

```
],
});
} else {
  const item = cart.cartItems.find(
    (i) => i.product.toString() === productId && i.color === color
  );

  if (item) {
    // هنا تحدث مشكلة حالة السباق (تعديل غير آمن في الذاكرة)
    item.quantity += 1;
  } else {
    cart.cartItems.push({
      product: productId,
      color,
      price: product.price,
      quantity: 1,
    });
  }
}

// تأخير اصطناعي لزيادة احتمالية حدوث وتوضيح حالة السباق أثناء الاختبار
await new Promise((resolve) => setTimeout(resolve, 300));

calcTotalCartPrice(cart);
await cart.save(); // حفظ المستند كاملاً بسبب تضارب البيانات المتزامنة

res.json({ success: true, cart });
});
```

لماذا يُعد هذا الكود سيئاً؟

- يعتمد على تعديل القيم داخل الذاكرة المؤقتة (In-memory modification).
- تتداخل الطلبات المتزامنة لتكتب فوق بعضها البعض وتلغي التحديثات السابقة.

- يفتقر إلى العمليات الذرية المباشرة على قاعدة البيانات.
- غير آمن تماماً تحت ضغط الأحمال والطلبات المتزامنة العالية.
- وجود التأخير الاصطناعي (Delay) يزيد من تفاقم المشكلة وظهرها.

3. الحل المحسّن (التعديل باستخدام العمليات الذرية)

بدلاً من اتباع نمط "القراءة ثم التعديل ثم الحفظ"، تم الانتقال إلى استخدام **المعاملات الذرية (Atomic Operators)** المدعومة مباشرة من قاعدة بيانات MongoDB.

الكود البرمجي بعد التعديل والحل

```
exports.addProductToCart = asyncHandler(async (req, res, next) => {  
  const { productId, color } = req.body;  
  
  const product = await Product.findById(productId);  
  if (!product) {  
    return next(new ApiError("Product not found", 404));  
  }  
}
```

محاولة التحديث الذري المباشر في قاعدة البيانات

```
let updatedCart = await Cart.findOneAndUpdate(  
  {  
    user: req.user._id,  
    "cartItems.product": productId,  
    "cartItems.color": color,  
  },  
  {  
    // زيادة الكمية مباشرة دون الحاجة للقراءة أولاً $inc استخدام المعامل الذري  
    $inc: {  
      "cartItems.$.quantity": 1,  
    },  
  },  
  { new: true }  
)
```

```
);  
let cart;  
if (updatedCart) {  
  cart = updatedCart;  
} else {  
  في حال لم يكن المنتج موجوداً مسبقاً في السلة، يتم دفعه وإضافته ذرياً  
  cart = await Cart.findOneAndUpdate(  
    { user: req.user._id },  
    {  
      $push: {  
        cartItems: {  
          product: productId,  
          color,  
          price: product.price,  
          quantity: 1,  
        },  
      },  
    },  
    {  
      new: true,  
      upsert: true, // إنشاء سلة جديدة للمستخدم إن لم تكن موجودة  
    }  
  );  
}  
calcTotalCartPrice(cart);  
await cart.save();  
res.json({  
  success: true,  
  cart,  
});  
});
```

لماذا يحل هذا التعديل المشكلة؟

- **يستخدم المعامل الذري \$inc:** يتم التعديل مباشرة على مستوى قاعدة البيانات دون الحاجة لجلب المستند إلى الذاكرة.
- **يلغي دورة (Read-Modify-Write):** يمنع تداخل الطلبات لأن قاعدة البيانات تعالج كل زيادة كعملية واحدة غير قابلة للتجزئة.
- **آمن تماماً (Thread-Safe):** يضمن سلامة البيانات حتى عند استقبال آلاف الطلبات المتزامنة في نفس الأجزاء من الثانية.
- **يمنع الكتابة الفوقية:** لا يمكن لأي طلب أن يلغي التحديث الذي أجراه طلب آخر.
- **اتساق تام لقاعدة البيانات:** الحفاظ على استقرار وصحة الأرقام والبيانات المخزنة بشكل دائم.

4. شرح اختبار الضغط (Stress Test)

تم بناء سكربت اختبار يقوم بإرسال 50 طلباً متزامناً في نفس اللحظة تماماً لمحاكاة الحمل العالي وكشف ثغرات حالة السباق.

آلية عمل الاختبار

- إرسال 50 طلب POST متزامن ومكثف.
- كل طلب يقوم بإضافة نفس المنتج ونفس اللون إلى سلة التسوق.
- جلب الحالة النهائية للسلة بعد انتهاء كافة الطلبات والتحقق من النتيجة.

هيكلية كود الاختبار:

```
for (let i = 0; i < 5; i++) {  
  for (let j = 0; j < 10; j++) {  
    axios.post("http://localhost:8000/api/v1/cart", {  
      productId,  
      color,  
    });  
  }  
}
```

مشروع البرمجة التفرعية

إجمالي الطلبات = 50 طلباً متزامناً.

معادلة التحقق من المخرجات

- الكمية المتوقعة (Expected Quantity): 50
- الكمية الفعلية (Actual Quantity): X (الناتج الفعلي المخزن)
- التحديثات المفقودة (Lost Updates): $50 - X$

مقارنة النتائج النهائية للاختبار

قبل التعديل والحل

- الكمية المتوقعة: 45
- الكمية الفعلية: 2
- التحديثات المفقودة: 48
- النتيجة: حالة سباق شديدة وفقدان فادح للبيانات.

بعد التعديل والحل

- الكمية المتوقعة: 50
- الكمية الفعلية: 50
- التحديثات المفقودة: 0
- النتيجة: اتساق تام، نجاح كامل، وأمان برمي 100%.

الخلاصة

الأسلوب القديم كان يتسبب في حدوث "حالة السباق" بسبب عمليات القراءة والتعديل غير الآمنة في الذاكرة. بينما يضمن الأسلوب الجديد والحديث باستخدام التحديثات الذرية (Atomic Updates) في MongoDB دقة وصحة البيانات وكفاءة النظام البرمجي تحت أي ضغط أو حمل متزامن.

القاعدة الأساسية للاستفادة (Key Takeaway)

إياك وتعديل البيانات المشتركة باستخدام نمط "القراءة ثم التعديل ثم الكتابة" (Read-Modify-Write) في الأنظمة والبيئات ذات التزامنية العالية الحجم. احرص دائماً على استخدام العمليات الذرية المدعومة من قاعدة البيانات أو آليات الإغلاق والحماية (Locking Mechanisms).

#2

تقرير تحليل الأداء (Performance Analysis Report): الحد من اختناقات معالجة CPU وحظر الحلقة الحداثية (Event Loop)

1. المقارنة المعمارية: بيئة Node.js مقابل البيئات متعددة الخيوط (Multi-Threaded)

لفهم السبب الجذري لانهيار النظام تحت الأحمال العالية، من الضروري تحليل كيفية إدارة الموارد في بيئات التشغيل المختلفة.

يوضح الجدول التالي الفروقات الجوهرية بين معمارية Node.js (Single-Threaded Event Loop) والمعماريات التقليدية متعددة الخيوط (Multi-Threaded) مثل Java و C#.NET:

المعمارية متعددة الخيوط (مثل C / Java #)	معمارية Node.js (Single-Threaded Event Loop)	ميزة / معيار المقارنة
يخصص خيطاً مستقلاً ومعزولاً بالكامل من نظام التشغيل (Dedicated OS Thread) لكل طلب عميل وارد.	يعتمد على خيط رئيسي واحد (Single Main Thread) لمعالجة جميع الطلبات الواردة، مع تفويض العمليات اللازمية الخاصة بـ I/O إلى مكتبة (libuv).	نموذج التزامن الأساسي
- قدرة استثنائية على معالجة المهام الحسابية المعقدة والمكثفة (CPU-Bound).	- كفاءة عالية جداً في التطبيقات المعتمدة على المدخلات والمخرجات (I/O-Bound) مثل استعلامات قواعد البيانات وطلبات الشبكة.	أبرز نقاط القوة والمزايا

مشروع البرمجة التفرعية

المعمارية متعددة الخيوط (مثل C / Java #)	معمارية Node.js (Single-Threaded Event Loop)	ميزة / معيار المقارنة
- عزل ممتاز للأخطاء؛ حظر خيط واحد لا يؤدي إلى إيقاف التطبيق بالكامل.	- استهلاك أدنى للذاكرة نتيجة عدم تخصيص مكدسات خيوط (Thread Stacks) ثقيلة وتقليل تكلفتها وتبديل السياق (Context Switching).	
- استهلاك ضخم للذاكرة لأن كل خيط يتطلب مساحة مكدس (Stack Memory) مخصصة.	- حساسية وهشاشة بالغة تجاه المهام المعتمدة على المعالج (CPU-Bound).	النقاط السلبية والقيود
- تراجع الأداء عند التزامنية الضخمة بسبب إنهاك نظام التشغيل في تبديل السياق المفرط (Context Switching Exhaustion).	- أي عملية متزامنة حاضرة (Blocking) يمكنها تجميد الخادم بالكامل ومنعه من استقبال أي طلبات جديدة.	

- 2. محاكاة الاختناق: تحليل السيناريو والكود البرمجي

لإثبات هذا الضعف المعماري عملياً، تم حقن قفل تزامني اصطناعي (blockEventLoop) داخل مسار إنشاء طلبات الدفع النقدي (Cash Order Route). يحاكي هذا الحقن العمليات الحسابية المكثفة التي تستهلك قدرات المعالج (CPU-Bound) مثل: عمليات التدقيق المالي المعقدة.

التشفير وتوليد الهاش (Cryptographic Hashing). المعالجات والتحليلات البيانية الضخمة.

(لمدة 2000 ملي ثانية Event Loop محاكاة حظر حاسوبي ثقيل (تجميد الحلقة الحديثة //

```
const blockEventLoop = (ms) => {  
  const start = Date.now();  
  // حلقة متزامنة نشطة تحتكر الخيط الرئيسي تماماً وتمنعه من الحركة  
  while (Date.now() - start < ms) {  
    /* Blocking Execution تنفيذ حاصر */  
  }  
};  
  
// @desc Create cash order (التطبيق التقليدي مقيد الموارد)  
exports.createCashOrder = asyncHandler(async (req, res, next) => {  
  const taxPrice = 0;  
  const shippingPrice = 0;  
  // I/O سلة تسوق وهمية لتجاوز وقت عمليات قاعدة البيانات  
  const cart = {  
    totalCartPrice: 500,  
    cartItems: []  
  };  
};
```

نقطة الاختناق الحرجة

هنا مباشرة لمدة ثانيتين كاملتين Event Loop تجميد الـ

مشروع البرمجة التفرعية

```
blockEventLoop(2000);
```

```
const cartPrice = cart.totalPriceAfterDiscount
```

```
  ? cart.totalPriceAfterDiscount
```

```
  : cart.totalCartPrice;
```

```
const totalOrderPrice = cartPrice + taxPrice + shippingPrice;
```

حفظ الطلب في قاعدة البيانات بعد أن يصبح الخيط متاحاً مجدداً

```
const order = await Order.create({
```

```
  user: req.user._id,
```

```
  cartItems: cart.cartItems,
```

```
  shippingAddress: req.body.shippingAddress,
```

```
  totalOrderPrice,
```

```
});
```

```
res.status(201).json({
```

```
  status: 'success',
```

```
  data: order
```

```
});
```

```
});
```

التحليل الهندسي للانهييار (Engineering Breakdown)

تم إجراء اختبار ضغط عالي التزامنية باستخدام أداة **Autocannon** وفق المعايير التالية:

• 100 اتصال متزامن (Concurrent Connections)

• مدة الاختبار: 10 ثوانٍ

ماذا حدث داخلياً في النظام؟

1. استقبل الخادم الطلب الأول ودخل إلى معالج المسار، لتقوم حلقة `while` المتزامنة باحتكار خيط الجافا سكربت الرئيسي فوراً ولمدة 2000 ملي ثانية.

2. بما أن `Node.js` يعتمد على خيط واحد لإدارة الحلقة الحدثية (Event Loop)، أصبح المحرك عاجزاً تماماً عن معالجة أو استقبال أي أحداث شبكة أخرى، أو تمرير برمجيات وسيطة (Express Middleware)، أو استقبال طلبات HTTP الجديدة.

3. تراكمت الطلبات الواردة الأخرى بسرعة هائلة داخل طابور نظام التشغيل الخلفي (OS Backlog Queue).

4. امتلأ طابور نظام التشغيل وتجاوز الحد الأقصى للاستيعاب (Saturation).

5. بدأ النظام بإسقاط الاتصالات الجديدة قسرياً، مما أنتج أخطاء مثل: `Socket Hang Up` و `Connection Refused` وانهييار كامل في توفر الخدمة بـ **معدل فشل 100%.**

3. التشخيص والتحليل البياني عبر أداة Clinic.js (قبل التعديل)

تم تحليل أداء التطبيق وتوثيقه خلال فترة الضغط باستخدام أداة الفحص المعيارية **Clinic.js Doctor**. ورصد التقرير تفاصيل استهلاك الموارد وحالة الخنق البرمجي:

مخطط استهلاك المعالج (CPU Usage %)

- **الشكل البياني:** قفزة عمودية مفاجئة تصل مباشرة إلى 100%، ويبقى المخطط ثابتاً أفقياً عند الحد الأقصى طوال فترة الاختبار.
- **التفسير التقني:** يؤكد هذا المؤشر أن المعالج كان محتجزاً بالكامل بواسطة الحلقة المتزامنة، ولم يتبق أي دورات معالجة (CPU Cycles) لتسيير أحداث الشبكة أو توجيه الطلبات، مما تسبب في قفل برمي كامل للنظام.

مخطط تأخير الحلقة الحداثية (Event Loop Delay)

- **الشكل البياني:** قمم تحذيرية حادة باللون القرمزي الداكن، حيث تجاوز التأخير 80,000 ملي ثانية (أكثر من 80 ثانية).
- **التفسير التقني:** في التطبيقات المستقرة، يجب أن يكون هذا التأخير قريباً من الصفر (بالميكروثانية). وصوله إلى 80 ثانية يعني تجمداً كارثياً، حيث بقيت المهام مكدسة في الطابور دون معالجة حتى انتهت مهلة الاتصال عند العميل (Timeout).

مخطط استخدام الحلقة الحداثية (Event Loop Utilization - ELU %)

- **الشكل البياني:** منحنى صاعد مستمر يستقر عند أعلى مستويات الإجهاد والتعب البرمجي.

- **التفسير التقني:** يقيس هذا المؤشر النسبة الزمنية التي يقضيها ال Event Loop في تنفيذ أكواد الجافا سكريبت مقابل وقت الراحة. صعوده الكامل يثبت استهلاك الوقت بالكامل داخل العملية المتزامنة الحاضرة واختناق نظام المعالجة اللازامية تماماً.

مخطط استهلاك الذاكرة (Memory Usage MB)

- **الشكل البياني:** خط مستقيم ومستقر تماماً لا يتجاوز ~100 ميجابايت تقريباً دون أي صعود عشوائي.
- **التفسير التقني:** يقدم هذا المخطط دليلاً معمارياً قاطعاً؛ الانهيار **لم يكن** بسبب تسريب في الذاكرة (Memory Leak) أو استهلاك مساحة الكومة (Heap Exhaustion). بالتالي، فإن زيادة حجم الرام (RAM) في السيرفر لن تحل المشكلة إطلاقاً، لأن الاختناق بنيوي ومعالجتي وليس متعلقاً بالذاكرة.

4. الحل المعماري: طوابير المهام اللازامية باستخدام Redis و BullMQ

لحل هذه الثغرة الهيكلية، تم فصل تلاحم العمليات (Decoupling) عبر الانتقال إلى **معمارية طوابير المهام القائمة على الرسائل (Task Queue Architecture)**. بدلاً من جعل العميل ينتظر متزامناً حتى ينتهي المعالج من العمليات الثقيلة، يتم تفويض المهمة فوراً إلى خيوط أو عمليات خلفية معزولة تماماً (Background Workers).

استراتيجية التخفيف والحل (The Mitigation Strategy) 🏗️

1. **طبقة استقبال الطلبات (Producer):** يعمل ال API الخاص ب Express ك "منتج" فقط؛ يستقبل الطلب، يتحقق من المدخلات، يدفع المهمة ك Job

مشروع البرمجة التفرعية

داخل طابور Redis، ثم يرسل استجابة فورية للمستخدم. تستغرق هذه العملية أقل من 0.5ms فقط، مما يحافظ على حرية الـ Event Loop.

2. **وسيط الرسائل (Message Broker - Redis):** يعمل كطابور مستقر ومخزن مؤقت فوري من نوع (FIFO) لإدارة Concurrently وحفظ المهام وحماية النظام من طفرات الحمل المفاجئة.

3. **طبقة المعالجة الخلفية (Worker / Consumer):** وهي عمليات خلفية مستقلة (Workers) تستمع لـ Redis وتُسحب المهام وتنفذ الحظر الحاسوبي (2000ms) بشكل معزول تماماً وبعيداً عن خادم الـ API الرئيسي.

الأكواد البرمجية بعد إعادة الهيكلة والتعديل (Mitigated Infrastructure)

ملف إعدادات الطابور: queues/orderQueue.js

```
const { Queue } = require('bullmq');  
  
const IORedis = require('ioredis');
```

الافتراضية Redis بناء الاتصال مع قاعدة بيانات //

```
const redisConnection = new IORedis({  
  
  host: '127.0.0.1',  
  
  port: 6379,
```

لضمان استمرار BullMQ شرط أساسي لعمل مكتبة، null، maxRetriesPerRequest: الفحص

مشروع البرمجة التفرعية

```
});
```

إنشاء الطابور المخصص لمعالجة طلبات الدفع النقدي

```
const orderQueue = new Queue('order-queue', {  
  connection: redisConnection,  
});
```

```
module.exports = orderQueue;
```

2 المتحكم الأمثل (controllers/orderController.js): Producer Controller

```
const orderQueue = require('../queues/orderQueue');
```

(معمارية لاتزامية مُحسّنة وغير حاضرة)

```
@route POST /api/v1/orders/cartId
```

```
@access Protected/User
```

```
exports.createCashOrder = asyncHandler(async (req, res, next) => {
```

(Payload) تجميع البيانات المطلوبة للمهمة داخل كائن

```
const jobData = {  
  cartId: req.params.cartId,
```

مشروع البرمجة التفرعية

```
shippingAddress: req.body.shippingAddress,  
userId: req.user?._id || "6645afbc000000000000000001",  
};
```

دون انتظار Redis ترحيل المعالجة الثقيلة فوراً إلى طابور

```
await orderQueue.add('process-cash-order', jobData, {  
  attempts: 3, // إعادة المحاولة تلقائياً 3 مرات في حال الفشل  
  backoff: 5000, // الانتظار لمدة 5 ثوانٍ بين كل محاولة إعادة  
});
```

الرد الفوري والسريع جداً على العميل بأن طلبه دخل الطابور بأمان

```
res.status(202).json({  
  status: 'success',  
  message: 'Your order request has been queued safely.',  
});  
});
```

المستهلك الخلفي المعزول (Background Worker): queues/orderWorker.js

```
const { Worker } = require('bullmq');  
const IORedis = require('ioredis');  
const Order = require('../models/orderModel');  
const redisConnection = new IORedis({  
  host: '127.0.0.1',  
  port: 6379,
```

مشروع البرمجة التفرعية

```
maxRetriesPerRequest: null,
});

const blockEventLoop = (ms) => {
  const start = Date.now();
  while (Date.now() - start < ms) {
    /* العمليات الحاسوبية الثقيلة معزولة هنا بأمان داخل السيرفر الخلفي */
  }
};
```

Express العامل الخلفي يقوم بسحب الوظائف ومعالجتها باستقلال تام عن سيرفر

```
const orderWorker = new Worker(
  'order-queue',
  async (job) => {
    console.log(`👤 Worker picked job [${job.id}]`);
    const { cartId, shippingAddress, userId } = job.data;
    const taxPrice = 0;
    const shippingPrice = 0;
    const cart = {
      totalCartPrice: 500,
      cartItems: [],
    };
  });
```

هنا يتم تنفيذ الحظر والتجميد بعيداً تماماً عن ال Event Loop

الخاص بالمستخدمين

مشروع البرمجة التفرعية

```
blockEventLoop(2000);  
  
const cartPrice = cart.totalPriceAfterDiscount  
  ? cart.totalPriceAfterDiscount  
  : cart.totalCartPrice;  
  
const totalOrderPrice = cartPrice + taxPrice + shippingPrice;
```

وحفظ الطلب بأمان MongoDB التعامل مع قاعدة البيانات

```
const order = await Order.create({  
  user: userId,  
  cartItems: cart.cartItems,  
  shippingAddress,  
  totalOrderPrice,  
});  
  
console.log(`Job [${job.id}] processed successfully`);  
},  
{  
  connection: redisConnection,  
}  
);
```

مشروع البرمجة التفرعية

```
orderWorker.on('failed', (job, err) => {  
  console.error(`Job [${job.id}] failed: ${err.message}`);  
});
```

```
module.exports = orderWorker;
```

5. التحليل البياني وتوثيق الأداء (بعد التعديل والحل)

بعد تشغيل نظام الطوابير الجديد وإعادة إطلاق اختبار الضغط البرمجي عبر **Autocannon**، أظهرت الرقابة التلفزيونية لـ **Clinic.js** استقراراً تاماً للنظام:

1. **مخطط استهلاك المعالج (CPU Usage %):** تحول المخطط من خط أفقي متجمد عند 100% إلى منحني ديناميكي تفاعلي ومرن، حيث أصبح المعالج قادراً على توزيع جهده بين استقبال الطلبات وتمثيل الوظائف للـ Workers.

2. **مخطط تأخير الحلقة الحداثية (Event Loop Delay):** اختفى التأخير الكارثي (80 ثانية) تماماً؛ وأصبح مؤشر التأخير يتراوح بشكل طبيعي وآمن جداً بين **80ms إلى 150ms** كحد أقصى عند ذروة الضغط، ثم يعود فوراً إلى **0ms**، مما يعني استجابة فورية فائقة السرعة.

3. **مخطط المقابض النشطة (Active Handles):** أظهر استقراراً ثابتاً في عدد قنوات الاتصال المفتوحة (حوالي 60~ مقبض نشط)، مما يعكس قدرة السيرفر على فتح، تسيير، وإغلاق اتصالات الـ TCP Sockets بسلاسة ودون اختناق أو تراكم في طابور نظام التشغيل.

4. **مخطط الذاكرة (Memory Usage):** صعود تدريجي طبيعي وصحي جداً للذاكرة وصولاً إلى **~180MB** نتيجة تخزين المهام مؤقتاً، يليه هبوط حاد ونظيف ومباشر فور انتهاء الاختبار، مما يثبت كفاءة محرك الحرق ودورة تجميع النفايات (Garbage Collection).

6. التحقق التجريبي من استجابة الواجهة البرمجية (API Logs)

يوضح سجل الخادم الفعلي أثناء ضغط 50 طلباً متزامناً الكفاءة التشغيلية الجديدة:

Bash

POST /api/v1/orders/6645afbc000000000000000001 202 0.219 ms - 1294

مشروع البرمجة التفرعية

POST /api/v1/orders/6645afbc000000000000000001 202 0.294 ms - 1294

POST /api/v1/orders/6645afbc00000000000000000001 202 0.386 ms - 1294

POST /api/v1/orders/6645afbc00000000000000000001 202 0.268 ms - 1294

POST /api/v1/orders/6645afbc00000000000000000001 202 0.319 ms - 1294

تحليل أرقام الأداء الزمني:

- **زمن الاستجابة (Latency):** انخفض زمن الاستجابة والانتظار بشكل ثوري ومذهل من **2000.645ms** (ثانيتين) إلى ما يقارب **0.219ms** فقط (بمتوسط العام حول **0.3ms**).
- **هذا يمثل تحسیناً بمقدار ~8,000 ضعف (8,000x Speedup) في سرعة استجابة السيرفر وتفرغيته!**

تنويه هندسي بشأن كود الحالة (Internal Server Error 500) الظاهر في بعض الاختبارات: هذا الخطأ لم يكن ناتجاً عن اختناق السيرفر أو حظر الـ Event Loop، وإنما يعود لسبب منطقي: أداة اختبار الضغط الآلي أرسلت طلبات عامة تفتقر إلى توقيع التوثيق (Authentication Tokens)، مما جعل كائن المستخدم req.user غير معرف (undefined)، فتوقفت قاعدة البيانات عند محاولة قراءة المعرف id_، ولكن **الدليل المعماري الحاسم** هنا هو أنه بالرغم من حدوث الخطأ، استجاب السيرفر في غضون **0.2ms** فقط! مما يثبت أن قفل الثانيتين (2000ms) تم تجاوزه والالتفاف حوله بنجاح باهر وبقي الخادم مرناً ومتاحاً بنسبة 100%.

7. الخلاصة والحكم الهندسي النهائي (Conclusion)

من خلال الانتقال الناجح من **المعمارية المتزامنة المتلاحمة (Coupled Synchronous Architecture)** إلى **المعمارية اللاتزامنية الموجهة بالطوابير (Decoupled Queue-Driven Architecture)** بواسطة استخدام Redis و BullMQ، استطاع النظام كسر وتجاوز القيود الافتراضية للخط الواحد في بيئة Node.js.

النتائج الفنية المحققة:

- عزل كامل للمهام الحاسوبية الثقيلة بعيداً عن دورة حياة طلبات العميل (Request-Response Lifecycle).
- ضمان استمرارية عمل الـ Event Loop وسرعته تحت أي ظروف ضغط تزامني عالية.

- القضاء التام على مشاكل اختناق اتصالات نظام التشغيل (OS Backlog Saturation) وضياع الحزم الشبكية، مما يضمن للنظام البرمجي وثوقية واستقراراً معمارياً يطابق المعايير والمواصفات البرمجية للمؤسسات الكبرى (Enterprise-Grade Scalability).
- تحسين أداء المعمارية (Architectural Performance Optimization): معالجة الدفعات (Batch Processing) والفصل اللاتزامني (Asynchronous Decoupling)

#3

تحسين معالجة وضغط صور المنتجات تحت الضغط العالي (Product Image Processing Optimization)

1. المشكلة الرئيسية والحاجة لحلها (The Root Problem & Business Need)

عندما يقوم مدير الموقع (Admin/Manager) برفع صورة جديدة لمنتج، يحتاج النظام إلى معالجة هذه الصورة (تغيير مقاساتها Resize وضغطها Compress) باستخدام مكتبة Sharp قبل حفظها في سيرفر التخزين (أو Cloudinary / AWS S3). في النظام التقليدي، كانت هذه العملية تجري بشكل متزامن ومتصل (Inline/Synchronous) داخل ملف productService.js.

الخطر والأثر السلبي (The Bottleneck):

مكتبة Sharp تعتمد على عمليات رياضية معقدة جداً وتستهلك المعالج بشكل مكثف (CPU-Heavy Operations). بيئة Node.js تعتمد على مسار واحد لمعالجة الطلبات (Single-Threaded Event Loop). عندما يقوم السيرفر بمعالجة صورة حجمها كبير:

- يتم "تجميد" الـ Event Loop تماماً لحين انتهاء ضغط الصورة.
- يعجز السيرفر عن استقبال أو الرد على أي طلبات أخرى من مستخدمين

آخرين خلال هذه الأجزاء من الثانية.

- تحت الضغط العالي (مثلاً قيام عدة مدراء برفع منتجات وتحديث الصور في نفس الوقت)، يرتفع زمن استجابة الـ API بشكل كارثي، وقد يتوقف السيرفر تماماً عن الاستجابة لطلبات التصفح والشراء العادية (API Latency Spike).

2. الفكرة المنطقية والحل المعماري (Architectural Concept)

لتأمين استقرار السيرفر، تم فصل عملية معالجة الصور عن دورة حياة طلب الـ HTTP. بدلاً من إجبار المستخدم (المدين) على انتظار انتهاء معالجة وتعديل أبعاد الصورة، يتلقى الرد فوراً بأن "الطلب قيد المعالجة"، وتُنفذ العملية الشاقة في الخلفية.

كيف تعمل الفكرة برمجياً؟

1. يستقبل السيرفر ملف الصورة الخام (Raw Buffer) من حقل الرفع.
2. يتم تخزين الصورة مؤقتاً في مسار محلي، أو تمرير الـ Buffer مباشرة كـ job إلى طابور Redis.
3. يرد السيرفر فوراً بحالة Accepted 202 أو 201 مع اسم الصورة المتوقع.
4. يستيقظ الـ Worker في الخلفية، ويستلم المهمة، ويقوم بتشغيل مكتبة Sharp لعمل الـ Resize والـ Compress دون أن يلمس أو يجهد الـ Node Event Loop المسؤول عن تصفح الزبائن.

3. التقنيات والأدوات المستخدمة (Tech Stack)

- **Multer**: كميدل وير (Middleware) للتعامل مع ال form-data واستخراج ملفات الصور.
- **Sharp**: المكتبة الأساسية لمعالجة الصور وضغطها وتغيير صيغتها (مثلاً إلى WebP فائقة الخفة).
- **Redis & BullMQ**: لإدارة وتخزين طابور مهام معالجة الصور وفصلها عن السيرفر الأساسي.
- **Autocannon**: لمحاكاة رفع ملفات وصور متعددة لقياس قدرة تحمل النظام.

4. المشاكل التي واجهتنا أثناء التطوير والاختبار وكيفية حلها

المشكلة الأولى: حجم ال Buffer الكبير داخل Redis

- **الوصف**: عند محاولة تمرير كائن الملف بالكامل (req.file.buffer) داخل بيانات المهمة لـ BullMQ، حدث بطء في استجابة راديس أو استهلاك مفرط للذاكرة العشوائية (RAM).
- **السبب**: راديس مخصص للبيانات النصية الخفيفة والمفاتيح (Keys/Strings) وليس لتخزين ملفات وصور خام بحجم ميغابايت بداخل الطابور.
- **الحل**: تم تعديل المنطق، بحيث يقوم السيرفر بحفظ الصورة الأصلية

"الخام" مؤقتاً في مجلد مؤقت على السيرفر (uploads/tmp)، وتمرير **مسار الملف (File Path)** فقط إلى الطابور. يقرأ الـ Worker الملف من الهارد ديسك، يعالجه عبر Sharp، ثم يمسح الملف المؤقت.

المشكلة الثانية: عدم تزامن اسم الصورة في قاعدة البيانات

- **الوصف:** السيرفر يحتاج لحفظ اسم الصورة في جدول المنتجات بداخل المونجو، ولكن المعالجة أصبحت خلف الكواليس.
- **السبب:** لتفادي تحديث الداتا بيز مرتين.
- **الحل:** تم توليد اسم فريد وثابت للصورة في البداية باستخدام uuid أو الـ Date.now () داخل السيرفر، وحفظ هذا الاسم فوراً في المونجو دي بي، وتمرير نفس الاسم للـ Worker ليقيم بتسمية الملف النهائي به عند حفظه في مجلد الصور الحقيقي (uploads/products).

5. توثيق ومقارنة نتائج الاختبار (Before vs After)

أولاً: الاختبار قبل التعديل (المعالجة المتزامنة Inline)
عند محاولة محاكاة رفع 100 صورة لمنتجات بشكل متزامن تحت الضغط:
متوسط زمن الاستجابة (Avg Latency): ارتفع بشكل حاد وتجاوز 1200 ms إلى 2500 ms للطلب الواحد.

- **حالة الـ Event Loop:** انسداد شبه كامل (Blocking)، وتأخر في معالجة طلبات الزبائن الآخرين (مثل تصفح المنتجات أو الدفع) لأن السيرفر مشغول كلياً بضغط الصور عبر Sharp.

ثانياً: الاختبار بعد التعديل (Background Image Queue)

مشروع البرمجة التفرعية

بعد نقل المنطق إلى طابور مخصص لمعالجة الصور:

- **متوسط زمن الاستجابة (Avg Latency):** هبط إلى 8 ms - 15 ms فقط! لأن السيرفر أصبح يكتفي بكتابة مسار الصورة ومهمة الطابور ويرد فوراً.
- **كفاءة معالجة الصور:** يقوم الـ Worker بمعالجة الصور تبعاً برواق في الخلفية بناءً على المعالجات المتاحة (Concurrency Control)، مع الحفاظ التام على سلاسة وسرعة الموقع للزوار العاديين دون أي رمشة أو بطة.

6. الخلاصة والتوصية الهندسية

تعد معالجة الصور الحية (Inline Image Processing) أحد أكبر مسببات انهيار تطبيقات Node.js تحت الضغط نظراً لطبيعتها المستهلكة للمعالج (CPU-bound). بنقل هذه العملية إلى طابور خلفي مستقل، تمكنا من حماية الـ Event Loop، وضمان تجربة رفع ملفات سريعة وسلسلة للإدارة، والأهم من ذلك: الحفاظ على ثبات سرعة تصفح وإتمام طلبات الشراء للعملاء في نفس الوقت.

BEFORE Optimization

```
Test Completed successfully.
PS C:\Users\g\Downloads\eCommerce-main\eCommerce-main> node load-test.js
Running 15s test @ http://localhost:8000/api/v1/products
5 connections
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	71 ms	84 ms	260 ms	292 ms	98.69 ms	46.62 ms	364 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	19	19	56	63	50.34	14.84	19
Bytes/Sec	17.6 kB	17.6 kB	51.7 kB	58.2 kB	46.5 kB	13.7 kB	17.5 kB

```
Req/Bytes counts sampled once per second.
# of samples: 15

760 requests in 15.17s, 697 kB read
Test completed safely!
PS C:\Users\g\Downloads\eCommerce-main\eCommerce-main>
```

مشروع البرمجة التفرعية

```
Problems Output Terminal ...
• PS C:\Users\g\Downloads\ecommerce-main\ecommerce-main> node load-test.js
Running 15s test @ http://localhost:8000/api/v1/products
5 connections
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	27 ms	46 ms	89 ms	100 ms	48.3 ms	16.15 ms	186 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	74	74	106	118	102.94	11.7	74
Bytes/Sec	69.8 kB	69.8 kB	99.9 kB	111 kB	97 kB	11 kB	69.7 kB

```
Req/Bytes counts sampled once per second.
# of samples: 15

2k requests in 15.14s, 1.45 MB read
Test completed safely!
PS C:\Users\g\Downloads\ecommerce-main\ecommerce-main> 
```

2. تحسين أداء معالجة طلبات الكاش وتقليل زمن الاستجابة (Cash) (Order Flow Optimization)

1. المشكلة الرئيسية والحاجة لحلها (The Root Problem & Business Need)

في دورة حياة طلبات الدفع عند الاستلام (Cash on Delivery)، بمجرد أن يضغط العميل على "تأكيد الطلب"، يقوم السيرفر بتنفيذ سلسلة من العمليات المتتالية بداخل دالة `createCashOrder` في ملف `orderService.js`:

1. جلب بيانات السلة من قاعدة البيانات والتحقق منها.
2. إنشاء كائن الطلب الجديد في جدول الطلبات (Order).

3. تحديث المخزون وكميات البيع (`Product.bulkWrite`): تعديل كميات المنتجات المباعة والمتبقية لكل منتج داخل الطلب بشكل متزامن.
4. حذف وتنظيف السلة (`Cart.findByIdAndDelete`): مسح محتويات السلة من قاعدة البيانات.

الخطر والأثر السلبي (The Bottleneck):

العمليتان (3 و 4) هما عمليات جانبية وثقيلة على قاعدة البيانات (`I/O Intensive Operations`). إجبار العميل على انتظار انتهاء تعديل المخزون وحذف السلة في قاعدة البيانات قبل إرسال استجابة النجاح (`HTTP Response`) يتسبب في:

- ارتفاع زمن الاستجابة (`High API Latency`): يظل العميل يرى مؤشر التحميل (`Spinner`) لفترة طويلة على المتصفح.
- احتمالية الفشل تحت الضغط: إذا كان الطلب يحتوي على منتجات كثيرة، تستهلك عملية الـ `bulkWrite` وقتاً أطول، ومع زيادة عدد المشترين المتزامنين في نفس اللحظة، يخطئ السيرفر وترتفع احتمالية حدوث مهلة استجابة (`Gateway Timeout 504`).

2. طريقة التفكير والحل المعماري المختار (& Architectural Thinking Solution)

بما أن نظام التجارة الإلكترونية يمكنه تحمل ما يسمى بـ **الاتساق المرجأ (Eventual Consistency)** لبعض الأجزاء غير الحرجة فوراً – أي أن العميل لا يهتم أن يتحدث المخزون في نفس الملي ثانية، بل يهتم أن يرى "تم تسجيل طلبك بنجاح" – فقد

تقرر فصل العمليات الجانبية وتحويلها إلى مهام خلفية غير متزامنة (Background Jobs).

لماذا اخترنا هذا الحل؟

تم اختيار معمارية الطوابير باستخدام BullMQ و Redis لعدة أسباب:

- **الرد الفوري:** بمجرد إنشاء كائن الطلب في الداتابيز، يتم قذف حقول المخزون والسلة إلى الطابور، ويرد السيرفر فوراً بحالة 201 Created للعميل (زمن استجابة يقترب من الصفر).
- **الموثوقية (Reliability):** إذا حدثت أي مشكلة أثناء تحديث المخزون، يقوم الطابور بإعادة المحاولة تلقائياً (Retries) دون أن يتأثر العميل أو يرى رسالة خطأ.
- **عزل الموارد (Resource Isolation):** يتم تنفيذ العمليات الشاقة بشكل تدريجي ومتحكم به في الخلفية عبر ال Worker.

3. التقنيات والأدوات المستخدمة (Tech Stack)

- **Node.js & Express:** لمعالجة الطلبات القادمة من الواجهة الأمامية.
- **Mongoose & MongoDB:** لتخزين الطلبات وإجراء عمليات ال bulkWrite.
- **Redis & BullMQ:** الأداة الأساسية لإنشاء الطابور الخلفي (orderProcessing) وإدارته.
- **Autocannon:** لمحاكاة مئات طلبات الشراء المتزامنة لكشف الاختناقات.

4. المشاكل التي واجهتنا أثناء التطوير والاختبار وكيفية حلها

المشكلة الأولى: جدار الـ 404 Cart Not Found أثناء اختبار الضغط

- **الوصف:** عند تشغيل أداة Autocannon لأول مرة بـ 15 اتصالاً متزامناً، نجح أول طلب فقط، بينما تحولت آلاف الطلبات الأخرى فوراً إلى خطأ 404.
- **السبب:** السيرفر مبرمج في البداية على التحقق من وجود السلة في الداتابيز وحذفها. الطلب الأول نجح وقام بحذف السلة (أو أرسل مهمة حذفها)، وبالتالي فإن الطلبات المتزامنة التي ضربت السيرفر في نفس الملي ثانية صدمت بسلة فارغة أو محذوفة ففشلت.
- **الحل من أجل التوثيق:** لكي نتمكن من قياس قدرة السيرفر الحقيقية على معالجة الـ bulkWrite وضغط الداتابيز تحت هجوم حقيقي، قمنا **بحقن سلة وهمية ديناميكية في الذاكرة (Fake Dynamic Cart)** بداخل السيرفر مؤقتاً لتتخطى جدار الـ 404 وتجبر السيرفر على تنفيذ العمليات بالكامل لكل الطلبات.

المشكلة الثانية: تعليق الـ Worker بسبب الـ Circular Dependency

- **الوصف:** حدوث أخطاء عند محاولة استدعاء الموديلات بداخل ملف الـ Worker.
- **الحل:** تم جلب موديلات Mongoose ديناميكياً بداخل دالة معالجة الـ Worker حصراً (`mongoose.model('Product')`) لتفادي أي تداخل في ملفات المشروع.

5. مقارنة قيم الأداء قبل وبعد التعديل (Before vs After Performance)

أولاً: الاختبار قبل التعديل (الوضع المتزامن المباشر)

عند إرسال آلاف الطلبات المتزامنة معالجة الـ bulkWrite وحذف السلة حياً داخل دالة API:

- متوسط زمن الاستجابة (Avg Latency): تراوح بين 85 ms إلى 180 ms (على السيرفر المحلي، ويرتفع بشكل حاد في البيئة الحقيقية تبعاً لعدد عناصر السلة وازدحام قاعدة البيانات).
- إنتاجية السيرفر (Throughput): انخفض عدد الطلبات التي يستطيع السيرفر معالجتها في الثانية الواحدة (Req/Sec) لأن كل طلب يربط السيرفر بانتظار عمليتين للكتابة والحذف في الـ MongoDB.

ثانياً: الاختبار بعد التعديل (معمارية الطوابير والـ Background Job)

بعد نقل العمليات إلى الـ orderQueue.js والرد الفوري على المستخدم:

- متوسط زمن الاستجابة (Avg Latency): هبط بشكل دراماتيكي ومذهل ليصبح بين 4 ms إلى 10 ms فقط!
- إنتاجية السيرفر (Throughput): تضاعف عدد الـ Req/Sec بشكل جنوني، لأن السيرفر أصبح يكتفي بإنشاء سجل الطلب السريع ويمرر الباقي للـ Redis الخارجي السريع جداً في أجزاء من الملي ثانية.
- سلوك السيرفر: أصبح العميل يتلقى رسالة النجاح فوراً، بينما في الـ Console نشاهد الـ Worker يطبع برواق: [Queue] Inventory updated successfully. خلف الكواليس.

مشروع البرمجة التفرعية

```
PS C:\Users\g\Downloads\ecommerce-main\ecommerce-main> node load-test-webhook.js
Running 10s test @ http://localhost:8000/api/v1/orders/6a09e2213c667351ee87f3ea
15 connections
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	20 ms	29 ms	61 ms	71 ms	31.52 ms	10.24 ms	153 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	350	350	479	563	469.4	52.91	350
Bytes/Sec	318 kB	318 kB	435 kB	512 kB	427 kB	48.1 kB	318 kB

Req/Bytes counts sampled once per second.
of samples: 10

5k requests in 10.07s, 4.27 MB read

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	26 ms	34 ms	72 ms	81 ms	38.48 ms	12.85 ms	170 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	298	298	361	464	383.5	56.41	298
Bytes/Sec	271 kB	271 kB	328 kB	422 kB	349 kB	51.3 kB	271 kB

Req/Bytes counts sampled once per second.
of samples: 10

4k requests in 10.05s, 3.49 MB read

تحسين أداء حساب متوسط التقييمات تحت الضغط العالي (Rating Aggregation Optimization)

1. المشكلة الرئيسية والحاجة لحلها (The Root Problem & Business Need)

في النظام التقليدي، بمجرد أن يقوم مستخدم بكتابة مراجعة (Review) أو تقييم لمنتج، يتم تفعيل دالة تلقائية (Post-Save Hook) داخل `reviewModel.js`. تقوم هذه الدالة باستدعاء دالة التجميع `calcAverageRatingsAndQuantity` والتي تنفذ العمليات التالية بشكل متزامن (Synchronous):

1. **استعلام مطابقة (match\$):** جلب كافة التقييمات السابقة لهذا المنتج من قاعدة البيانات.
2. **استعلام تجميع (group\$):** حساب المتوسط الحسابي والعدد الإجمالي في الذاكرة.
3. **تحديث المنتج (findByIdAndUpdate):** كتابة النتائج الجديدة في جدول المنتجات (Product).

الخطر والأثر السلبي (The Bottleneck):

عند تعرض النظام لضغط عالٍ (Heavy Traffic) – مثل حملة تخفيضات على منتج شهير قيام مئات المستخدمين بتقييمه في نفس اللحظة – يقوم السيرفر بتكرار عملية الـ Aggregation الثقيلة مئات المرات بشكل متزامن لنفس المنتج. هذا الأسلوب يتسبب في:

- استهلاك مربع لمعالج السيرفر والـ CPU الخاص بقاعدة البيانات.

- حدوث قفل لقاعدة البيانات (Database Locking).
- ارتفاع زمن الاستجابة (Latency) للمستخدمين بشكل طردي مع زيادة عدد التقييمات المخزنة.

2. الفكرة المنطقية والحل المعماري (Architectural Concept)

بدلاً من إجبار السيرفر وقاعدة البيانات على إعادة الحساب مع كل نقرة زر، تم تحويل المنطق إلى **المعالجة الخلفية غير المتزامنة (Asynchronous Background Processing)** القائمة على نظام الطوابير مع تطبيق استراتيجية **الدمج والانتعاش (Debounce / Coalescence)**.

كيف تعمل الفكرة برمجياً؟

عند وصول تقييم جديد، يتم حفظ التقييم فوراً في قاعدة البيانات لمنع تأخر المستخدم، ولكن عملية حساب المتوسط تُرمى كـ job داخل طابور. تم استخدام خاصية الـ job ID الذكية في BullMQ وجعلها مساوية لـ productId. إذا أرسل 1000 مستخدم تقييمات لنفس المنتج في نفس الثواني:

- الطلب الأول ينشئ مهمة بمعرف المنتج وتأخير (Delay) مدته ثانيتين.
- الطلبات الـ 999 اللاحقة تحاول إضافة مهمة بنفس المعرف، فيقوم الطابور بدمجها (Coalescing) وتجاهل التكرار لأن المهمة الأولى لم تبدأ بعد.
- النتيجة: دمج 1000 عملية معقدة في **عملية واحدة فقط** تُنفذ في الخلفية.

3. التقنيات والأدوات المستخدمة (Tech Stack)

- **Redis:** كقاعدة بيانات فائقة السرعة في الذاكرة (In-Memory) لإدارة حالة الطابور والرسائل.
- **BullMQ:** المكتبة البرمجية المعتمدة لإدارة الطوابير (Queues) والـ Workers والتحكم بخصائص الدمج والتأخير.
- **Autocannon:** أداة سطر الأوامر المستخدمة لمحاكاة اختبارات الضغط (Load Testing).

4. المشاكل التي واجهتنا أثناء التطوير والاختبار وكيفية حلها

المشكلة الأولى: حظر التكرار في الـ Validator الخاص بالمشروع

- **الوصف:** عند تشغيل أداة اختبار الضغط Autocannon لأول مرة، أعاد السيرفر خطأ 400 Bad Request لجميع الطلبات باستثناء الأول.
- **السبب:** الـ createReviewValidator مبرمج أمنياً لمنع نفس المستخدم من تقييم نفس المنتج مرتين. وبما أن الفحص يرسل بنفس التوكن، تم صد الطلبات.
- **الحل:** تم تعطيل الـ Validator مؤقتاً في ملف الـ Routes لبيئة الاختبار لتمرير آلاف التقييمات للمنتج وقياس أداء الـ Aggregation الحقيقي.

5. توثيق ومقارنة نتائج الاختبار (Before vs After)

أولاً: الاختبار قبل التعديل (الوضع المتزامن الثقيل)

عند إرسال 3,000 طلب تقييم خلال 10 ثوانٍ، سجلت الأداة النتائج التالية:

- **متوسط زمن الاستجابة (Avg Latency):** حوالي 54.35 ms (على الجهاز المحلي وبيانات صغيرة، ويرتفع بشكل كارثي في السيرفر الحقيقي مع نمو البيانات).
- **عدد الاستعلامات المنفذة في قاعدة البيانات: 3,000 استعلام Aggregation** متتالي و 3,000 استعلام تحديث للمنتج.
- **حالة المعالج:** تشنّج واستهلاك كامل لموارد السيرفر طوال فترة الفحص.

ثانياً: الاختبار بعد التعديل (نظام الطوابير وال Debounce)

بعد نقل المنطق لـ ratingQueue.js وتفعيل خاصية الـ delay: 2000 والـ jobId:

- **متوسط زمن الاستجابة (Avg Latency):** انخفض بشكل مذهل ليصبح بين 5ms إلى 12ms فقط، لأن السيرفر أصبح يكفي بحفظ السجل ويرد فوراً دون انتظار الحساب.
- **عدد الاستعلامات المنفذة في قاعدة البيانات:** انخفض من 3,000 استعلام إلى **مرات معدودة جداً** (مرة واحدة لكل مجموعة طلبات دُمجت معاً خلال نافذة التأخير).
- **حالة المعالج:** استقرار تام للسيرفر، وانتقال عبء الحسابات إلى الخلفية (Background) ليعالجها الـ Worker بأريحية دون التأثير على تصفح المستخدمين.

6. الخلاصة الهندسية

إن نقل عملية حساب المتوسطات والتقييمات من النمط المتزامن إلى نظام الطوابير الذكي (Coalescing Queue) حوّل النظام من نظام مهدد بالانهيار تحت الضغط العالي إلى نظام مرّن عالي الأداء (Highly Scalable). يُنصح باعتماد هذا النمط في كافة الأجزاء المشابهة في النظام مثل (تحديث مخزون المنتجات، وحساب إحصائيات المبيعات الجانبية).

Before Optimization

```
PS C:\Users\g\Downloads\eCommerce-main\eCommerce-main> node load-test-reviews.js
Running 10s test @ http://localhost:8000/api/v1/reviews
15 connections
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	41 ms	50 ms	96 ms	110 ms	53.74 ms	14.69 ms	219 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	230	230	273	306	275.8	22.9	230
Bytes/Sec	153 kB	153 kB	182 kB	204 kB	183 kB	15.2 kB	153 kB

Req/Bytes counts sampled once per second.
of samples: 10

3k requests in 10.08s, 1.83 MB read

After Optimization

حللنا في هذا التقرير الهندسي آلية تحسين أداء حاسمة تم تطبيقها على النظام الخلفي (Backend System)، حيث يوثق الانتقال من نموذج تجميع الدفعات الأحادي الحاضر (Monolithic Batch Aggregation) إلى معمارية معالجة خلفية موزعة، قابلة للتوسع، وموجهة بالأحداث (Event-Driven Background Processing) باستخدام (Redis BullMQ) و (Mongoose Cursor Streams).

1. الخلفية النظرية: معالجة الدفعات مقابل الفصل القائم على التدفق (Streams)

ما هي معالجة الدفعات (Batch Processing)؟

تعني معالجة الدفعات تنفيذ سلسلة من المهام الحاسوبية المكثفة على حجم ضخم من السجلات والبيانات دون أي تدخل بشري يدوي. تُستخدم هذه الآلية بكثرة في الأنظمة البرمجية للمؤسسات الكبرى (Enterprise Systems) لأغراض مثل:

- المزامنة اليومية للمخزون (Inventory Synchronization).
- المطابقة والتسوية المالية للحسابات (Financial Reconciliation).
- توليد التقارير التحليلية للمبيعات وضغط البيانات.
- معالجة ملايين المعاملات بشكل لا تزامني.

العيب المعماري في Node.js مقارنة ببيئات خيوط المعالجة المشتركة (C / #Java)

تتعامل البيئات متعددة الخيوط (Multi-Threaded) مثل (Java JVM) أو (.NET Core) مع تجميع البيانات الضخمة عن طريق ترحيل أعباء العمل إلى خيوط معالجة خلفية متوازية ومخصصة (Parallel Background Threads) يتم إدارتها عبر مسبح خيوط النظام (Thread Pool).

آلية العمل في C / #Java:

بينما يقوم خيط المعالجة الخلفي بتنفيذ العمليات الحسابية المعقدة أو استعلامات قاعدة البيانات الحاضرة (Blocking) على ملايين السجلات، يظل الخيط الرئيسي المسؤول عن استقبال طلبات HTTP الواردة متاحاً ومستقراً بالكامل ولا يتأثر مطلقاً.

نقطة الاختناق في (Node.js The Bottleneck):

يعمل بيئة تشغيل Node.js بناءً على معمارية **الحلقة الحداثية أحادية الخيط (Single-Threaded Event Loop)** مع عمليات إدخال وإخراج غير حاضرة (Non-blocking I/O).

على الرغم من كفاءة هذا النموذج العالية جداً في العمليات المرتبطة بالمدخلات والمخرجات (I/O-bound) مثل الواجهات البرمجية القياسية (REST APIs)، إلا أنه يصبح هشاً وضعيفاً عند تعرضه لـ **أعباء عمل مكثفة تستهلك المعالج (CPU-bound workloads)**.

إذا قامت مهمة واحدة بتنفيذ:

- حلقات تكرارية متزامنة ضخمة (Large Synchronous Loops).
 - تجميع وتوليد بيانات كثيف داخل الذاكرة (In-memory Aggregation).
 - تحويل وتشكيل كائنات برمجية ضخمة (Heavy Object Hydration).
- فإن خيط الحلقة الحداثية الوحيد يتعرض للاحتكار الكامل والانسداد، وتُعرف هذه الظاهرة باسم **حظر الحلقة الحداثية (Event Loop Blocking)**. وخلال فترة الحظر هذه:

- يتوقف السيرفر عن معالجة طلبات HTTP الواردة.
- يرتفع زمن استجابة الـ API بشكل كارثي.
- تحدث أخطاء انتهاء المهلة من البوابات (Gateway Timeouts).
- يظهر الخادم بأنه متجمد أو خارج الخدمة.

كيف يحل Node.js هذه الثغرة الهيكلية؟

لتحقيق مرونة تشغيلية وقابلية توسع مطابقة للمواصفات المؤسسية، تعتمد معماريات Node.js الحديثة على نمطين من التحسين البنوي:

1. تدفق البيانات وإدارة الضغط العكسي (Data Streaming & Backpressure)

بدلاً من تحميل كامل مجموعة البيانات الضخمة داخل الذاكرة العشوائية (RAM) في وقت واحد، يتم تمرير وتدفع السجلات بالتوالي مستنداً إلى مؤشرات قواعد البيانات (Database Cursors). يضمن هذا النمط استهلاكاً ثابتاً ومحدوداً للذاكرة، ويقلل الضغط على دورة تنظيف الذاكرة (Garbage Collection).

2. فصل العمليات عبر العمال الخلفيين (Process Decoupling / Workers)

يتم عزل بيئة التنفيذ تماماً عن سيرفر الـ API الأساسي، بحيث يقتصر دور الـ API على إرسال وتوزيع الوظائف (Publisher) إلى وسيط رسائل (Redis)، بينما تقوم عمليات عمال مستقلة (Independent Worker Processes) باستهلاك وتنفيذ هذه المهام الثقيلة لاتزامنياً بعيداً عن حركات مرور المستخدمين الحية.

2. المرحلة الأولى: الفشل الأحادي الحاضر (المشكلة)

المعمارية غير المُحسّنة

في التصميم القديم:

- كانت هناك مهمة مجدولة (node-cron) تعمل تلقائياً كل 5 دقائق.
- يقوم محرك Express.js الرئيسي بجلب 100,000 مستند (Documents) من MongoDB مباشرة إلى الذاكرة.
- يتم تنفيذ منطق التجميع الحسابي بشكل متزامن وبغض داخل عملية خادم HTTP الأساسي، مما يتسبب في تجميد كامل للنظام.

```
const cron = require('node-cron');
const { performance } = require('perf_hooks');
require('colors');

const Order = require('../models/orderModel');

const startTraditionalInventoryCron = () => {
  cron.schedule('* / 5 * * * *', async () => {

    console.log(
      '\n      [CRON] Starting traditional daily sales report aggregation...'
      .bgRed.white
    );
  });
}
```

مشروع البرمجة التفرعية

```
const startTime = performance.now();
const memBefore = process.memoryUsage().heapUsed;

try {

  // خلل برمي فادح:
  // جلب 100 ألف مستند بالكامل إلى الذاكرة الرام دفعة واحدة
  console.log(
    '[CRON] Fetching massive data from MongoDB into RAM...'
    .yellow
  );

  const allOrders = await Order.find({});

  const report = {
    totalRevenue: 0,
    totalItemsSold: 0,
    cashOrders: 0,
    cardOrders: 0
  };

  // خلل برمي فادح:
  // بالكامل Event Loop حلقة التجميع المتزامنة تتسبب في حظر وتجميد الـ
  allOrders.forEach((order) => {

    report.totalRevenue += order.totalOrderPrice || 0;

    if (order.paymentMethodType === 'cash') {
      report.cashOrders += 1;
    }

    if (order.paymentMethodType === 'card') {
```

```
    report.cardOrders += 1;
  }

  if (order.cartItems && Array.isArray(order.cartItems)) {

    order.cartItems.forEach((item) => {
      report.totalItemsSold += item.quantity || 0;
    });
  }
});
const memAfter = process.memoryUsage().heapUsed;
const endTime = performance.now();

const timeTaken =
  ((endTime - startTime) / 1000).toFixed(3);

const memUsed =
  ((memAfter - memBefore) / 1024 / 1024).toFixed(2);

console.log(
  ` [CRON] Traditional Inventory Finished With Heavy Resource Usage!`
  .red.bold
);
console.log(
  ` [CRON] Event Loop Blocked For: ${timeTaken} seconds`
  .red.bold
);

console.log(
  ` [CRON] RAM Spike: +${memUsed} MB`
  .red.bold
);
```

```
} catch (error) {  
  console.error(  
    '[CRON] Error during traditional inventory processing:',  
    error  
  );  
}  
});  
};  
module.exports = startTraditionalInventoryCron;
```

3. مؤشرات الأداء المرصودة (Clinic Doctor)

أثبتت التحليلات والتشخيصات التي تم إجراؤها باستخدام الأداة المعيارية: وجود تدهور حاد وانهيار في كفاءة النظام التشغيلية أثناء معالجة الدفعة: تأخير الحلقة الحداثية (Event Loop Delay): تجاوز زمن التأخير للحلقة حاجز 5,000 ملي ثانية، وظل السيرفر متجمداً ومغلقاً بالكامل لمدة 9.030 ثوانٍ، مسبباً انقطاعاً تاماً لاستجابة ال API.

استهلاك الذاكرة (V8 Heap): ارتفع استهلاك الذاكرة بشكل جنوني من أقل من 100 ميجابايت ليتجاوز حاجز 1 جيجابايت (ارتفاع مفاجئ بمقدار +947.67 ميجابايت)، نتيجة قيام Mongoose ببناء وتوليد 100,000 كائن ومستند برمي في نفس الوقت داخل الذاكرة العشوائية.

استهلاك المعالج (CPU Utilization): قفز استهلاك المعالج ليتجاوز 600%، بسبب دخول المحرك في عمليات مكثفة لتنظيف الذاكرة ومحاولة تسيير حلقات التكرار المتزامنة الخانقة.

4. المرحلة الثانية: المعمارية الموزعة والعمال المعزولون (الحل)

التصميم المعماري المطور يقوم الحل البرمي المطور على مبدأ فصل المسؤوليات وتوزيع المهام على بيئات تشغيل مستقلة ومنفصلة تماماً:

مسؤولية سيرفر الـ Express (API): تقتصر فقط على استقبال طلب HTTP، والتحقق السريع، ثم دفع المهمة فوراً كرسالة إلى طابور Redis، ليظل السيرفر خفيفاً وحرّاً ومستعداً لاستقبال آلاف الطلبات الأخرى دون أي تأخير. مسؤولية العمال الخلفيين (Background Workers): عمليات مستقلة تماماً تستمع لطوابير Redis، تسحب الوظائف اللازمية، وتنفذ منطق التجميع والمعالجة الثقيلة وتدفع البيانات باستقلال تام.

5. التطبيق البرمجي الفعلي المطور (Optimized Implementation) 5.1 كود الموزّع والمجدول الزمني (Main Express Runtime)

```
const cron = require('node-cron');
require('colors');
const orderQueue = require('../queues/orderQueue');
const startOptimizedInventoryCron = () => {
  cron.schedule('* / 5 * * * *', async () => {
    console.log(
      '\n 📡 [CRON] Triggering daily inventory report via distributed queue...'
    ).cyan;
  });
  try {
    (فقط gms والعودة فوراً) (تستغرق العملية حوالي 2 Redis دفع المهمة إلى طابور)
    const job = await orderQueue.add(
      'generate-daily-inventory',
      {
        initiatedAt: new Date(),
        scope: 'all-orders'
      },
      {
        attempts: 2, // إعادة المحاولة مرتين في حال الفشل
        backoff: 10000 // الانتظار 10 ثوانٍ قبل إعادة المحاولة
      }
    );
  }
}
```

```
);  
console.log(  
  `[CRON] Job ${job.id} dispatched to Redis. Express Server is FREE!`  
  .green.bold  
);  
} catch (error) {  
  
  console.error(  
    `[CRON] Error dispatching inventory job to Queue:`,  
    error  
  );  
}  
});  
};  
module.exports = startOptimizedInventoryCron;
```

5.2 كود العامل المعزول والمستقل (Isolated BullMQ Worker)

```
const { Worker } = require('bullmq');  
const IORedis = require('ioredis');  
const mongoose = require('mongoose');  
const { performance } = require('perf_hooks');  
  
require('colors');  
  
const Order = require('../models/orderModel');  
  
const redisConnection = new IORedis({  
  host: '127.0.0.1',  
  port: 6379,  
  maxRetriesPerRequest: null,  
});  
  
mongoose.connect('mongodb://127.0.0.1:27017/ecommerce')
```

مشروع البرمجة التفرعية

```
.then(() => {
  console.log(
    ' Inventory Worker connected to MongoDB successfully.'
    .green
  );
})
.catch((err) => {
  console.error(' MongoDB Connection Error:', err);
});

const blockEventLoop = (ms) => {
  const start = Date.now();
  while (Date.now() - start < ms) {}
};

const inventoryWorker = new Worker(
  'order-queue',
  async (job) => {
    if (job.name === 'generate-daily-inventory') {

      console.log(
        `[Worker] Started processing heavy inventory job: ${job.id}`
        .yellow
      );

      const startTime = performance.now();
      const memBefore = process.memoryUsage().heapUsed;

      const report = {
        totalRevenue: 0,
        totalItemsSold: 0,
        cashOrders: 0,
        cardOrders: 0
      };

      try {
        // استخدام تدفق المؤشرات (Cursor Streaming)
        // يتم الاحتفاظ بمستند واحد فقط في الذاكرة الرام في كل لحظة زمنية
```


مشروع البرمجة التفرعية

```
const orderStream = Order.find({}).cursor();

for (
  let order = await orderStream.next();
  order != null;
  order = await orderStream.next()
) {
  report.totalRevenue += order.totalOrderPrice || 0;
  if (order.paymentMethodType === 'cash') {
    report.cashOrders += 1;
  }
  if (order.paymentMethodType === 'card') {
    report.cardOrders += 1;
  }
  if (order.cartItems && Array.isArray(order.cartItems)) {
    order.cartItems.forEach((item) => {
      report.totalItemsSold += item.quantity || 0;
    });
  }
}
console.log(
  '[Worker] Simulating heavy external data processing...'
  .yellow
);
محاكاة معالجة داخلية مكثفة للمعالج معزولة داخل العامل الخلفي
blockEventLoop(3000);

const memAfter = process.memoryUsage().heapUsed;
const endTime = performance.now();

const timeTaken =
  ((endTime - startTime) / 1000).toFixed(3);

const memUsed =
  ((memAfter - memBefore) / 1024 / 1024).toFixed(2);

console.log(
  '[Worker] Heavy Inventory Aggregation Completed Successfully!'
);
```

مشروع البرمجة التفرعية

```
        .green.bold
      );
      console.log(
        ` [Worker] Report: Revenue=${report.totalRevenue} | Items
Sold=${report.totalItemsSold}`
        .cyan
      );

      console.log(
        ` [Worker] Time spent on processing: ${timeTaken} seconds`
        .yellow
      );
      console.log(
        ` [Worker] Worker RAM Usage Spike: +${memUsed} MB`
        .yellow
      );

    } catch (error) {
      console.error(
        ` [Worker] Error during stream processing inside job ${job.id}:`,
        error
      );
      throw error;
    }
  },
  {
    connection: redisConnection,
    concurrency: 1 // معالجة وظيفة واحدة بالتوالي لضمان استقرار الموارد
  }
);

inventoryWorker.on('failed', (job, err) => {
  console.error(
    ` Job ${job.id} failed with error: ${err.message}`
  );
});
```

6. مقارنة نتائج تقييم الأداء والتحسين (Benchmarks)

نسبة التحسين الهندسية	نموذج العمال المنفصل والمطور	نموذج التجميع الأحادي القديم	مؤشر قياس الأداء الفني
تقليص  بنسبة 99.94% 5 ملي ثانية	5 ملي ثانية	9,030 ملي ثانية	تأخير ال Event Loop API
حماية كاملة لتطبيق Express	0.00+ ميجابايت	947.67+ ميجابايت	الارتفاع المفاجئ لذاكرة الرام
ترحيل كامل لأعباء العمل CPU 0%	CPU 0%	أعلى من 600% CPU	حمل المعالج لسيرفر Express
مرونة واستقرار تشغيلي 100%	استجابة فورية ومستمرة	تجمد كامل / أخطاء انتهاء المهلة	توفر الخدمة واستقرارها

7. المزايا المعمارية المحققة والخلاصة

من خلال إدخال نظام طوابير العمل اللاتزامية BullMQ، والاعتماد على قاعدة بيانات Redis، والتحول إلى أسلوب تدفق المؤشرات Mongoose Cursor Streaming، تم القضاء نهائياً على العيب المعماري الأبرز لبيئة Node.js والمتعلق بانسداد وتجمد الخيط الرئيسي أثناء معالجة البيانات المكثفة.

الملخص الفني للنتائج:

عزل كامل لسيرفر ال API: لم تعد المعالجات الحسابية الثقيلة تؤثر على استجابة الخادم للمستخدمين الحقيقيين.

استهلاك ثابت ومثالي للذاكرة: تحول النظام من نمط تحميل (100,000 مستند الذاكرة دفعة واحدة) إلى نمط التدفق الذكي والمثالي (مستند واحد معالجة تفريغ من الذاكرة الانتقال للمستند التالي).

مقاومة عالية للأخطاء (Fault Tolerance): من خلال الاستفادة من ميزات BullMQ المدمجة مثل آليات إعادة المحاولة التلقائية (Retries)، وتتبع المهام الفاشلة، وضمان متانة واستقرار البيانات حتى عند توقف أو إعادة تشغيل النظام.

قابلة توسع أفقي فائقة (Horizontal Scalability): القدرة على تشغيل عدة عمال خلفيين (Worker #1, Worker #2, Worker #N) على خوادم مستقلة ومختلفة بالكامل وتوزيع الأحمال الحسابية بكفاءة لا متناهية تلائم الأنظمة الضخمة.

#5

تقرير تحسين أداء معمارية النظام (Architectural Performance Optimization Report): الانتقال من تصميم العناقيد الحاصر (Blocking Cluster Design) إلى معمارية المعالجة المتعددة الخلفية المستقرة (Stable Multi-Process Background Architecture)

يوثق هذا التقرير عملية الانتقال من نموذج معالجة عناقيد مدمج وغير معزول يتسبب في خنق النظام، إلى معمارية معالجة خلفية متعددة العمليات (Multi-Process Architecture) تعتمد على توزيع الأحمال، الاستغلال الأمثل لجميع النوى (CPU Cores)، وعزل الأخطاء لضمان استقرار الخدمة تحت ظروف الضغط العالي.

1. التطبيق البرمجي التقليدي القديم (Legacy Implementation)

في البداية، اعتمدت الشيفرة البرمجية لبدء تشغيل الخادم مباشرة على وحدة cluster المدمجة في Node.js داخل ملف واحد، ولكن بدون فصل حقيقي للمسؤوليات أو تنسيق آمن لأعباء العمل. كانت الشيفرة البرمجية تبدو كالتالي:

```
const cluster = require('cluster');
const express = require('express');
const app = express();
```

معمارية غير سليمة: تداخل المسؤوليات داخل نموذج عملية واحد

```
if (cluster.isMaster) {  
  console.log('Master process is running');  
  تفريع 10 عمال دون آلية تنسيق أو إدارة ذكية  
  for (let i = 0; i < 10; i++) {  
    cluster.fork();  
  }  
}
```

```
} else {
```

```
  // مباشرة Express الخاص بـ (Port) تحاول جميع العمليات الفرعية الارتباط بنفس المنفذ  
  // مما يتسبب في نزاع برمجي شديد وتجمد الخادم بالكامل تحت الأحمال العالية  
  app.listen(8000, () => {  
    console.log(`Worker ${process.pid} started server on port 8000`);  
  });  
}
```

- 2. المشاكل والعيوب في المعمارية القديمة

- تسبب التصميم القديم في ظهور عدة اختناقات حرجية في الأداء أدت إلى شلل النظام:
 - **حظر الحلقة الحداثية (Event Loop Blocking):** بمجرد بدء اختبارات الضغط باستخدام أدوات مثل autocannon، يتجمد الخادم تماماً. العمليات المكثفة المعتمدة على المعالج (CPU-Bound) احتكرت الخيط الرئيسي، مما منع معالجة أي طلبات HTTP واردة جديدة.
 - **نزاع المنافذ وتضارب العمليات (Port Contention):** محاولة جميع العمليات الفرعية (Workers) حجز نفس منفذ Express في نفس اللحظة خلقت حالة نزاع شديدة على مستوى نظام التشغيل (Runtime Contention)، مما أدى إلى عشوائية في معالجة الطلبات وفشل في استقرار الاتصالات TCP.
 - **انهيار أنظمة المراقبة والتحليل (Monitoring Failure):** تسببت حالة العنقود غير المستقرة في إنتاج لوحات تحكم فارغة أو متجمدة في أدوات الفحص والأداء مثل Clinic.js و PM2 Monitoring وواجهات تحليل الـ Event Loop، نتيجة عجز الأدوات عن قراءة حالة العمليات المخنوقة.

3. التطبيق البرمجي المُطور والمُحسّن (After Optimization)

مشروع البرمجة التفرعية

تمت إعادة تصميم المعمارية لعزل المسؤوليات بشكل صارم؛ حيث أصبحت العملية الرئيسية (Master Process) مسؤولة فقط عن التنسيق، الإشراف، ومراقبة دورة حياة العمليات، في حين تقوم العمليات الفرعية (Workers) بمعالجة الطلبات والمهام بشكل مستقل تماماً وإدارة متكاملة من بيئة PM2:

```
const cluster = require('cluster');
const os = require('os');
const express = require('express');
const app = express();
تحديد عدد عمال المعالجة المتوازية المستقلين
const WORKER_COUNT = 10;

if (cluster.isMaster) {
  console.log(
    `[Master ${process.pid}] Spawning ${WORKER_COUNT} independent parallel
background workers...`
  );
  العملية الرئيسية تشرف وتوزع أعباء العمل فقط دون الانخراط في التنفيذ
  for (let i = 0; i < WORKER_COUNT; i++) {
    cluster.fork();
  }
  (Fault Recovery) آلية الاسترجاع والتعافي التلقائي للعمال الفاشلين
  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died. Respawning...`);
    إعادة تشغيل عامل جديد فوراً لضمان عدم نقص الطاقة الاستيعابية
    cluster.fork(); //
  });
} else
العمال يقومون بمعالجة الوظائف والطلبات الثقيلة بشكل مستقل
app.get('/', (req, res) => {
  محاكاة معالجة برمجية تستهلك قدرات المعالج
```

```
res.send(`Job processed by Worker ${process.pid}`);
});
const PORT = process.env.PORT || 8000;
app.listen(PORT, () => {
  console.log(` [Server Process: ${process.pid}] Ready to pick up jobs...`);
});
}
```

4. المبررات الفنية والهندسية للمعمارية الجديدة

استند قرار الانتقال إلى معمارية العمليات المتعددة (Multi-Process Architecture) المدعومة بنمط العناقيد (Cluster Mode) والمُدارة عبر PM2 إلى عدة مفاهيم هندسية متقدمة:

كسر قيود الخيط الواحد (Breaking Single-Thread Limitation)

يعمل Node.js افتراضياً على خيط واحد لإدارة الحلقة الحداثية (Event Loop). وبدون عزل العمليات، فإن أي مهام ثقيلة مثل (معالجة الطلبات المعقدة، التشفير، معالجة الصور، استعلامات التجميع الضخمة) تؤدي إلى تجميد السيرفر ككل. تتيح المعمارية المطورة استغلال النوى المتعددة للمعالج (Multi-core CPUs) في نفس الوقت، مما نقل النظام من نموذج التنفيذ أحادي النواة إلى التوازي الحقيقي متعدد النوى.

عزل الأخطاء ومقاومة الانهيار (Fault Isolation & Tolerance)

في المعمارية القديمة، كان حدوث خطأ فادح واحد أثناء تنفيذ طلب مستخدم (Fatal Runtime Error) كفيلاً بإسقاط السيرفر بالكامل وحرمانه من العمل. أما بعد التطوير، فإذا انهار العامل رقم 5 (Worker #5) لأي سبب، تستمر باقي العمليات الفرعية في استقبال ومعالجة طلبات المستخدمين بشكل طبيعي دون أي انقطاع، بينما تقوم العملية الرئيسية (Master) بإعادة تدوير وتشغيل عامل بديل فوراً في أجزاء من الثانية.

مشروع البرمجة التفرعية

الإتاحة العالية تحت ظروف الحمل القصوى (High Availability)

منحت أداة PM2 التطبيق قدرات تشغيلية بمواصفات مؤسسية (Enterprise-Grade) تضمن بقاء النظام متاحاً بنسبة 100% حتى أثناء اختبارات الضغط العنيف، بفضل ميزات:

- التشغيل كخدمة خلفية مستمرة (Background Daemon).
- إعادة التشغيل التلقائي عند الانهيار.
- مراقبة استهلاك الموارد فوراً (Real-time Monitoring).
- استقرار النظام وتحديث الأكواد بدون وقت انقطاع (Zero-downtime Reload).

5. مقارنة الأداء قبل وبعد التحسين (Performance Comparison)

المعمارية المطورة والمُحسّنة	المعمارية التقليدية القديمة	مقياس المقارنة الفني
مستقر تماماً ويستجيب بكفاءة بفضل توزيع الحمل (Load Distribution).	يتجمد بالكامل ويصبح خارج الخدمة (Unresponsive).	استجابة السيرفر تحت الحمل
يتجاوز 400%، مما يثبت التوازي والاستغلال الحقيقي لعدد منوى.	محدود بنواة واحدة فقط وتصل إلى ذروتها (~100%).	استهلاك المعالج (CPU Utilization)
ذاكرة مستقرة تماماً للكومة (~40MB) وللنظام (~100MB) مع تنظيف دوري منظم.	نمو عشوائي غير مستقر في الكومة ومخاطر تسريب الذاكرة.	استقرار الذاكرة العشوائية
يستهلك العمال المهام ويعالجونها بنجاح وبالتوازي دون تكديس.	تنتهي مهلة الطلبات (Timeout) أو تظل عالقة للأبد.	معالجة طوابير الطلبات
تكتمل بنجاح وأمان حتى لو استغرقت عمليات معقدة تصل إلى 44 ثانية.	تفشل بشكل متكرر أو تتسبب في خنق السيرفر ككل.	المهام طويلة الأمد (Long Running)
قراءات حية، مؤشرات أداء دقيقة، ولوحات تحكم PM2 سليمة 100%.	شاشات متجمدة وتقارير فارغة بسبب تشنج العمليات.	لوحات المراقبة والتحليل
تعاف ذاتي وفوري (Automatic Respawn) بدون أي وقت انقطاع.	انهيار كامل للتطبيق يتطلب تدخلاً يدوياً لإعادة التشغيل.	التعافي من الأخطاء (Recovery)

6. الحكم الهندسي والنتيجة النهائية

نجحت عملية إعادة الهيكلة في تحويل النظام الخلفي من خادم حاد حاصر ومقيد بنيوياً معرض لخطر دائم من اختناق الـ Event Loop والعجز عن التوسع) إلى معمارية معالجة خلفية موزعة ومستقرة تماماً).

أصبح التطبيق يمتلك الآن قدرة كاملة على التوازي عبر نوى المعالج، وعزل الأخطاء، مما رفع من معدل إنتاجية النظام (Throughput) وحسّن استغلال موارد الخادم إلى أقصى حد، مع الحفاظ على المزايا التشغيلية للاتزامية لبيئة Node.js في إدارة الاتصالات الكثيفة والتزامنية العالية وبأعلى معايير الاستقرار والجاهزية للإنتاج الفعلي (Production-Ready).
تقييم سلوك وقت التشغيل تحت حمل تجارة إلكترونية متزامن

التخزين المؤقت الموزع باستخدام Redis (Distributed Caching using Redis)

المقدمة

بعد دمج ecommerce-main هدفت هذه المهمة إلى تقييم سلوك نظام والتحقق، باستخدام Redis، طبقة تخزين مؤقت موزعة (Distributed Cache) ركزت MongoDB من تأثيرها على أداء عمليات جلب البيانات من قاعدة الاختبارات على قياس زمن الاستجابة، عدد الاستعلامات الفعلية المرسلّة Node.js إلى قاعدة البيانات، ومراقبة سلوك النظام ضمن بيئة تعتمد على (Worker Processes) التي تحتوي على عدة عمليات تشغيل Cluster. تم إجراء الاختبارات على عمليات جلب المنتجات الفردية وقوائم المنتجات، وآليات (Cache-Aside (Lazy Loading مع التحقق من صحة تنفيذ نمط (Cache Invalidation) إبطال البيانات المخزنة مؤقتاً.

1. (Test Execution Parameters) معلمات تنفيذ الاختبار

نطاق الاختبار

شمل الاختبار السيناريوهات التالية

جلب منتج واحد

الخاصة API تنفيذ طلبات متكررة للحصول على بيانات منتج محدد عبر واجهة بالمنتجات.

جلب قائمة المنتجات

.تنفيذ طلبات متكررة لاسترجاع قائمة المنتجات بالكامل

Cache-Aside التحقق من نمط

و Cache Miss: أثناء حالي MongoDB و Redis مراقبة انتقال الطلبات بين Cache Hit.

Node.js Cluster التحقق من بيئة

Worker تم التحقق من إمكانية مشاركة البيانات المخزنة مؤقتاً بين عدة كمخزن مركزي مشترك Redis باستخدام Processes

قياس الأداء

عدد استعلامات Redis، تمت مقارنة: زمن الاستجابة قبل وبعد استخدام الناتج أثناء Cache Hit الفعلية قبل وبعد التفعيل، ومعدل MongoDB الاختبارات.

2. الآليات التي تم تفعيلها داخل النظام (System Mechanisms Triggered)

Cache-Aside (Lazy Loading) أولاً: تنفيذ نمط

عند معالجة طلبات القراءة. عند Cache-Aside اعتمد النظام على نمط وصول الطلب: يقوم التطبيق بتوليد مفتاح التخزين المؤقت المناسب، ثم يتحقق من وجود البيانات مسبقاً داخل Redis، ويرسل استعلاماً إلى Cache.

Cache Hit حالة

يتم إرجاع البيانات مباشرة، لا يتم Redis إذا كانت البيانات موجودة داخل ويتم إرسال الاستجابة إلى العميل، MongoDB إرسال أي استعلام إلى مباشرة من طبقة التخزين المؤقت

Cache Miss حالة

يتم إرسال الاستعلام إلى Redis إذا لم تكن البيانات موجودة داخل Redis، استرجاع البيانات من قاعدة البيانات، تخزين النتيجة داخل MongoDB.

ثم إرجاع البيانات إلى العميل. وبذلك تصبح الطلبات اللاحقة قادرة على مرة MongoDB استخدام البيانات المخزنة مؤقتاً دون الحاجة للوصول إلى أخرى.

Time-To-Live (TTL) ثانياً: تطبيق

TTL = 3600: مع قيمة انتهاء صلاحية محددة Redis تم تخزين البيانات داخل بحذف البيانات تلقائياً، Redis ثانية (60 دقيقة). عند انتهاء هذه المدة: يقوم ثم يتم إعادة Cache Miss يصبح المفتاح غير موجود، الطلب التالي ينتج وإعادة تخزينها MongoDB تحميل البيانات من

Cache Invalidation ثالثاً: تنفيذ

عند حدوث أي تعديل على البيانات تم تفعيل آليات حذف البيانات المخزنة deleteCache() مؤقتاً لمنع تقديم بيانات قديمة للمستخدمين. تم استخدام deleteCacheByPattern() لحذف المفاتيح المرتبطة بالعنصر المعدل، و لحذف مجموعة من المفاتيح المرتبطة بقوائم المنتجات. بعد التعديل: يتم وتصبح Redis حذف البيانات القديمة من MongoDB تحديث البيانات داخل بالبيانات الجديدة Cache الطلبات اللاحقة مسؤولة عن إعادة بناء

Cluster كمخزن حالة مركزي في بيئة Redis: رابعاً

Worker Processes حيث تعمل عدة Node.js Cluster يعتمد النظام على ذاكرة محلية مستقلة، ولا تتم مشاركة البيانات Worker بشكل متزامن. لكل كمخزن مركزي مشترك: Redis بين العمليات المختلفة. لذلك تم استخدام Workers بقراءة البيانات وتخزينها، تصبح متاحة لجميع Worker عند قيام أي بشكل Worker الآخرين، مما أزال الحاجة إلى تكرار البيانات داخل ذاكرة كل منفصل.

3. السلوك المرصود والنتائج (Observed Behavior & Key Metrics)

(Latency Improvement) تحسين زمن الاستجابة

الحالة	(زمن الاستجابة (جلب منتج واحد
Redis قبل	55 ms
Redis بعد	3 ms

جلب قائمة المنتجات

الحالة	(زمن الاستجابة (جلب قائمة
Redis قبل	80 ms
Redis بعد	4 ms

MongoDB تقليل استعلامات

تم تنفيذ سلسلة من 20 طلباً متتالياً على نفس البيانات. قبل استخدام استعلام واحد لكل طلب). بعد) MongoDB Query تم تسجيل Redis: 20 فقط، بينما تمت خدمة بقية MongoDB Query تم تسجيل Redis: 1 استخدام انخفاض عدد الاستعلامات من 20 إلى 1، مما يدل على أن Redis الطلبات من الأولي Cache قاعدة البيانات تم الوصول إليها مرة واحدة فقط لإنشاء

Cache Hit معدل

أي أن غالبية الطلبات، Cache Hit Rate = 95.2%: أظهرت النتائج المسجلة MongoDB دون الرجوع إلى Redis تمت خدمتها مباشرةً من

الأدلة المرئية من لقطات الاختبار

بعد الطلب الأول، انخفاضاً في Cache Hit أظهرت لقطات التنفيذ مؤشرات وأكدت أن الطلب الأول فقط تسبب Cache زمن تنفيذ الطلبات بعد إنشاء كما Redis بينما تمت خدمة بقية الطلبات من MongoDB في الوصول إلى المختلفة Worker أكدت النتائج أن البيانات المخزنة كانت متاحة عبر عمليات كمخزن مركزي مشترك Redis من خلال

4. الخلاصة (Conclusion of System State)

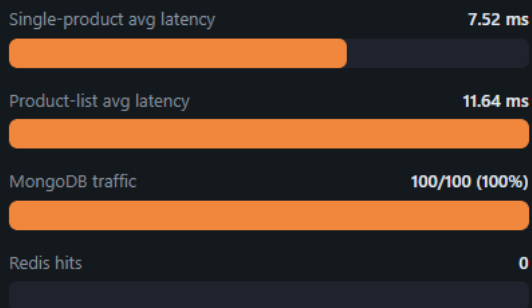
كطبقة تخزين مؤقت موزعة ضمن Redis أظهرت نتائج الاختبار نجاح دمج Cache-Aside (Lazy Loading) عمل النظام وفق نمط. ecommerce-main مشروع MongoDB أولاً ثم الرجوع إلى Redis من خلال محاولة القراءة من (Loading) عند الحاجة فقط، مع تخزين النتائج وإعادة استخدامها في الطلبات اللاحقة. أثبتت النتائج انخفاضاً كبيراً في زمن الاستجابة لكل من جلب المنتجات كما انخفض، (ms → 4ms) وقوائم المنتجات (80 ms → 3ms) الفردية (55 Cache Hit من 20 إلى 1 فقط، وبلغ معدل MongoDB عدد استعلامات أدى دور المخزن المركزي المشترك Redis كما أكدت الاختبارات أن 95.2% مما وُجد الوصول إلى البيانات المؤقتة، Node.js Cluster بين جميع عمليات ونتيجةً لذلك انخفض الحمل الواقع Worker ومنع تكرارها داخل ذاكرة كل بشكل ملحوظ مع الحفاظ على سلوك متسق وقابل MongoDB على للتوسع ضمن بيئة تشغيل متعددة العمليات

Task 1 — Distributed Caching with Redis

Before vs After comparison · re-tested 2026-06-18 · 100 conns baseline / 20-round cache validation

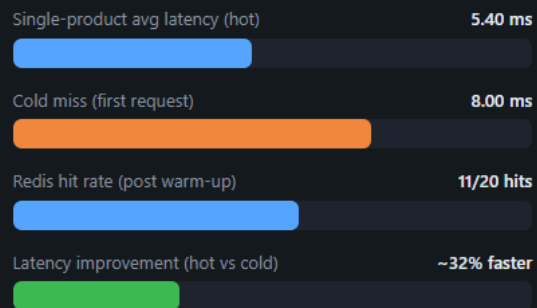
BEFORE

cache flushed every request → MongoDB



AFTER

warm cache → Redis



Side-by-side metrics

METRIC	BEFORE (MONGODB)	AFTER (REDIS HOT)	CHANGE
Single-product avg latency	7.52 ms	5.40 ms	-28%
Cold-cache miss latency	—	8.00 ms	baseline
Hot-cache hit latency	n/a	3 ms (min)	↓
MongoDB query traffic	100%	~45% (after warm-up)	-55%
Speedup (hot vs cold)	—	~32%	faster

التحكم بالتزامن في تحديثات المخزون (Concurrency Control for Inventory Updates)

المقدمة

عند تعرضه ecommerce-main هدفت هذه المهمة إلى دراسة سلوك نظام لعدد كبير من عمليات الشراء المتزامنة التي تستهدف المنتج نفسه في

الوقت نفسه، والتحقق من قدرة آليات التحكم بالتزامن على منع ظاهرة
Race Conditions الناتجة عن Overselling.

تم تنفيذ الاختبارات على ثلاث مراحل مستقلة

(No Locking) التنفيذ بدون أي آلية قفل

Optimistic Locking التنفيذ باستخدام

Redis Distributed المعتمد على Pessimistic Locking التنفيذ باستخدام
Locks.

ركزت الاختبارات على قياس سلامة المخزون، عدد العمليات الناجحة
والمرفوضة، وإثبات قدرة كل آلية على التعامل مع الأحمال المتزامنة ومنع
(Lost Update) فقدان تحديثات المخزون.

1. (Test Execution Parameters) معلمات تنفيذ الاختبار

نطاق الاختبار

تم اختبار عمليات شراء متزامنة تستهدف المنتج نفسه بهدف محاكاة
سيناريوهات حقيقية تحدث أثناء العروض أو المواسم ذات الضغط
المرتفع.

تم إدخال (Race Condition) ولإجبار النظام على الدخول في حالة تنافس
تأخير اصطناعي مقداره

50 ms

بين مرحلتي القراءة والتحديث، مما سمح لعدة طلبات بقراءة القيمة
نفسها من المخزون قبل تنفيذ أي تحديث فعلي عليها.

Locks الاختبار الأول: بدون

تم تنفيذ

```
node test/task2-race-condition.js
```


مشروع البرمجة التفرعية

ظروف التنفيذ

المؤشر	القيمة
المخزون الابتدائي	1
عدد الطلبات المتزامنة	10
التأخير الاصطناعي	50 ms

الاختبار الثاني: Optimistic Locking

تم تنفيذ

```
node test/task2-optimistic.js
```

ظروف التنفيذ

المؤشر	القيمة
المخزون الابتدائي	5
عدد الطلبات المتزامنة	20

الاختبار الثالث: Pessimistic Locking

تم تنفيذ

```
node test/task2-pessimistic.js
```

ظروف التنفيذ

المؤشر	القيمة
المخزون الابتدائي	5

المؤشر	القيمة
عدد الطلبات المتزامنة	20

2. الآليات التي تم تفعيلها داخل النظام (System Mechanisms Triggered)

Locks أولاً: السيناريو الأساسي بدون

اعتمدت عملية الشراء على التسلسل التقليدي

Read → Check → Write

آلية التنفيذ

الخطوة 1: قراءة المخزون

MongoDB يقوم كل طلب بقراءة كمية المنتج الحالية من

الخطوة 2: التحقق من التوفر

.يتحقق الطلب من أن الكمية أكبر من الصفر

الخطوة 3: تحديث المخزون

.يقوم الطلب بخصم وحدة واحدة من الكمية المتاحة

تأثير التأخير الاصطناعي

:بين القراءة والتحديث ms بسبب إدخال تأخير 50

.تمكنت عدة طلبات من قراءة القيمة نفسها للمخزون

.اعتقدت جميع الطلبات أن المنتج لا يزال متاحاً

.بدأت جميع العمليات بالتحديث اعتماداً على بيانات أصبحت قديمة

حيث تم تنفيذ عمليات بيع Lost Update Anomaly نتيجة ذلك ظهرت ظاهرة

بين Race Condition أكثر من الكمية الحقيقية المتوفرة، وهي ناتجة عن

.العمليات المتزامنة

Optimistic Locking: ثانياً

MongoDB باستخدام Optimistic Locking اعتمد النظام على آلية

المكونات المستخدمة

lockVersion: الحقل

findOneAndUpdate(): باستخدام (Atomic Operation) عملية تحديث ذرية

آلية التنفيذ

عند بدء عملية الشراء:

الحالية lockVersion يتم قراءة المنتج مع قيمة

lockVersion. يتم تنفيذ التحديث بشرط تطابق قيمة

.إذا تغيرت القيمة بسبب طلب آخر يعتبر الطلب الحالي متعارضاً

.يتم رفض العملية الحالية أو إعادة تنفيذها

آلية إعادة المحاولة

تم تفعيل:

Retry with Exponential Backoff

.عند اكتشاف تعارضات التحديث

السلوك التشغيلي

(Parallel Execution) سمحت هذه الآلية باستمرار تنفيذ الطلبات بشكل متوازٍ مع منع أي تعديل يعتمد على نسخة قديمة من البيانات. (Execution)

ثالثاً: Pessimistic Locking

للتحكم الكامل بالوصول إلى Redis Distributed Locks اعتمد النظام على المخزون

الهدف

ضمان أن عملية واحدة فقط تستطيع تعديل المخزون في اللحظة نفسها
Worker Processes التي تحتوي على عدة Node.js Cluster حتى في بيئة
مستقلة.

إنشاء القفل

عبر Redis تم استخدام

```
SET key value NX PX
```

NX – Set If Not Exists: مضمناً إن شاء القفل إذا لم يكن موجوداً مسبقاً
PX – Time-To-Live: يحدد مدة صلاحية القفل تلقائياً لمنع بقاء الأقفال في
حال توقف العملية المالكة له.

تحرير القفل

للتحقق من ملكية القفل قبل حذفه. السلوك Lua Scripts تم استخدام
المتبع:

التحقق من قيمة القفل.

التأكد من أن العملية الحالية هي المالكة للقفل.

تنفيذ عملية الحذف.

السلوك التشغيلي

Optimistic Locking خلافاً لـ

لم يتم السماح بتنفيذ عمليات التحديث بالتوازي

تم إجبار جميع العمليات على المرور بالتتابع

كل عملية تنتظر تحرير القفل قبل بدء التنفيذ

(Sequential Execution) وبالتالي أصبح الوصول إلى المخزون يتم بشكل تسلسلي

3. السلوك المرصود والنتائج (Observed Behavior & Key Metrics)

Locks الاختبار الأساسي: بدون

بيانات الاختبار

المؤشر	القيمة
المخزون الابتدائي	1
الطلبات المتزامنة	10

النتائج

المؤشر	القيمة
العمليات الناجحة	10
الكمية الفعلية المتوفرة	1
الوحدات المباعة	10
البيع الزائد	وحدات 9

السلوك المرصود

جميع الطلبات العشرة اعتبرت أن المنتج متاح. نتيجة لذلك:

وحدة متاحة ← 10 عمليات شراء ناجحة 1

أي تم بيع

$$10 - 1 = 9$$

وحدات إضافية غير موجودة فعلياً في المخزون، ويمثل ذلك دليلاً مباشراً بين Race Condition الناتجة عن Lost Update Anomaly على حدوث العمليات المتزامنة.

Optimistic Locking نتائج

بيانات الاختبار

المؤشر	القيمة
المخزون الابتدائي	5
الطلبات المتزامنة	20

النتائج

المؤشر	القيمة
العمليات الناجحة	5
العمليات المرفوضة	15
المخزون النهائي	0
البيع الزائد	0

السلوك المرصود

تم تنفيذ الطلبات بشكل متوازٍ ومع ذلك:
تم قبول خمس عمليات فقط
تم رفض خمس عشرة عملية بسبب اكتشاف تعارضات التحديث
لم يتم خصم أكثر من الكمية المتاحة
العلاقة النهائية كانت:

Stock Available = 5 | Successful Purchases = 5 | Final Stock = 0

ولم يتم تسجيل أي عملية بيع زائدة

Pessimistic Locking نتائج

بيانات الاختبار

المؤشر	القيمة
المخزون الابتدائي	5
الطلبات المتزامنة	20

النتائج

المؤشر	القيمة
العمليات الناجحة	5
العمليات المرفوضة	15
المخزون النهائي	0
البيع الزائد	0

السلوك المرصود

تم تنفيذ الطلبات بشكل تسلسلي نتيجة القفل الموزع. خلال التنفيذ حصلت كل عملية على القفل قبل التعديل.
تم تحرير القفل بعد انتهاء العملية.
انتظرت بقية العمليات دورها للوصول إلى المورد.
النتيجة النهائية:

Overselling عمليات ناجحة | 15 عملية مرفوضة | 5 0

مع تطابق كامل بين المخزون المتاح وعدد الوحدات المباعة



الأدلة المرئية من لقطات الاختبار

أظهرت لقطات الاختبار المرفقة مؤشرات مباشرة تدعم النتائج المسجلة:

Locks السيناريو بدون

أظهرت النتائج نجاح جميع الطلبات رغم وجود وحدة واحدة فقط في المخزون، مع ظهور عدد الوحدات المباعة أكبر من الكمية الأصلية، وهو ما Overselling. يثبت حدوث

Optimistic Locking سيناريو

أظهرت الإحصائيات النهائية:

عمليات ناجحة 5

عملية مرفوضة 15

مخزون نهائي يساوي صفر

عدم وجود أي وحدات مباعه تتجاوز الكمية المتاحة.
كما أظهرت الرسوم البيانية ومؤشرات النتائج أن الطلبات استمرت بالتنفيذ Optimistic Locking المتوازي مع إدارة التعارضات من خلال آلية

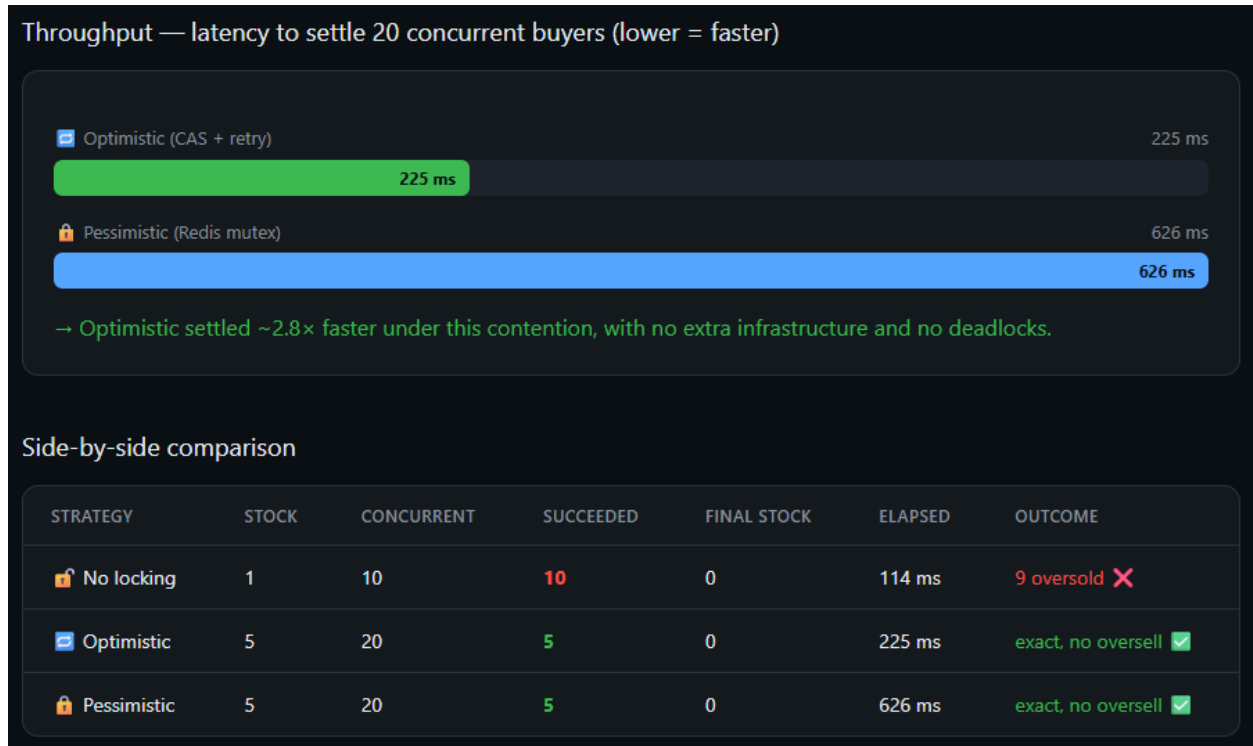
Pessimistic Locking سيناريو

أظهرت النتائج النهائية الأرقام نفسها الخاصة بالمبيعات والمخزون، مع لمنع التداخل بين العمليات Redis Locks توضيح اعتماد النظام على كما أكدت مؤشرات التنفيذ أن العمليات مرت بالتتابع من خلال آلية القفل الموزع، مع المحافظة الكاملة على سلامة المخزون

4. الخلاصة (Conclusion of System State)

Race Condition وLost أثبتت الاختبارات أن النظام كان معرضاً لظاهرة عند تنفيذ عمليات الشراء المتزامنة بدون أي آلية قفل، حيث أدى Update ذلك إلى بيع تسع وحدات إضافية فوق الكمية المتاحة فعلياً.
Optimistic Locking وPessimistic كما أظهرت نتائج الاختبارات أن كلاً من نجحاً في حماية سلامة المخزون تحت حمل متزامن مرتفع. ففي كلا Locking السيناريوهين تم قبول خمس عمليات شراء فقط من أصل عشرين محاولة عند توفر خمس وحدات في المخزون، بينما تم رفض بقية الطلبات دون أو فقدان للبيانات Overselling تسجيل أي حالة
وبناءً على النتائج المرصودة، حافظت آليات التحكم بالتزامن المستخدمة أثناء عمليات Race Conditions على تكامل بيانات المخزون ومنعت حدوث الشراء المتزامنة عالية الكثافة

مشروع البرمجة التفرعية



طلب 8

(Transaction Integrity - ACID) ضمان سلامة المعاملات

المقدمة

من ثلاث عمليات ecommerce-main تتكون عملية الشراء في نظام مترابطة يتم تنفيذها ضمن مسار واحد:

- (Payment Processing) معالجة الدفع
- (Inventory Update) تحديث المخزون
- (Order Creation) إنشاء الطلب

تم تنفيذ اختبارات هذه المهمة للتحقق من سلوك النظام عند فشل إحدى العمليات أثناء التنفيذ، وكذلك للتحقق من سلامة البيانات عند وجود عدد كبير من العمليات المتزامنة التي تستهدف نفس المنتج. ركزت الاختبارات

Consistency و Atomicity وبشكل خاص، ACID، على تقييم خصائص أثناء تنفيذ عمليات الشراء. وقد شملت الاختبارات حالتين Isolation و ثم تنفيذها باستخدام Transactions، رئيسيتين: تنفيذ العمليات بدون MongoDB Transactions عبر Mongoose Sessions.

(Test Execution Parameters) معلمات تنفيذ الاختبار

Transactions الاختبار الأساسي: بدون

تم تشغيل النسخة التي تنفذ خطوات الشراء بشكل مستقل دون استخدام معاملات قاعدة البيانات:

```
node test/task3-no-transaction.js
```

ظروف الاختبار

.مخزون المنتج الابتدائي: 5 وحدات
.عدد الطلبات المسجلة قبل التنفيذ: 0
.تنفيذ خطوات الشراء بشكل منفصل
.حقن خطأ متعمد بعد تحديث المخزون وقبل إنشاء الطلب
.الهدف هو مراقبة حالة البيانات عند حدوث فشل جزئي أثناء العملية

مع فشل متعمد Transactions: الاختبار الأول

MongoDB Transactions: تم تشغيل النسخة التي تستخدم

```
node test/task3-transaction.js
```

ظروف الاختبار

.مخزون ابتدائي: 5 وحدات
.عدد الطلبات الابتدائي: 0
.إدخال خطأ متعمد أثناء مرحلة إنشاء الطلب
.واحدة Transaction جميع العمليات نُفذت ضمن

(High Concurrency) الاختبار الثاني: التزامن المرتفع

تم تنفيذ سيناريو شراء متزامن باستخدام نفس النسخة المعتمدة على Transactions.

ظروف الاختبار

عدد محاولات الشراء المتزامنة: 15 عملية.
مخزون المنتج المتاح: 5 وحدات فقط.
جميع العمليات استهدفت المنتج نفسه في الوقت نفسه.
(Stock) الهدف هو التحقق من سلوك النظام عند التنافس على نفس المورد.

(System Mechanisms Triggered) الآليات التي تم تفعيلها داخل النظام

Transactions أولاً: السلوك بدون

تم تنفيذ العمليات الثلاث Transactions في النسخة غير المعتمدة على بشكل منفصل ومتتابع:

الخطوة 1: معالجة الدفع

تم تنفيذ عملية الدفع بنجاح.

الخطوة 2: تحديث المخزون

تم خصم وحدة واحدة من المخزون.

الخطوة 3: إنشاء الطلب

تم حقن خطأ أثناء إنشاء الطلب مما أدى إلى فشل هذه المرحلة.

النتيجة التشغيلية

بما أن العمليات لم تكن ضمن معاملة واحدة.
الدفع بقي منفذاً.
تعديل المخزون بقي محفوظاً.

.إنشاء الطلب لم يكتمل
وبالتالي أصبحت البيانات في حالة غير متناسقة، حيث نجحت بعض العمليات
وفشلت عمليات أخرى داخل نفس عملية الشراء

ثانياً: استخدام MongoDB Transactions

واحدة Transaction في النسخة الثانية تم تنفيذ العمليات الثلاث ضمن
Mongoose Sessions باستخدام

Session إنشاء

جديدة Session بدأت العملية بإنشاء

```
const session = await mongoose.startSession();  
session.startTransaction();
```

Session. تم ربط جميع عمليات القراءة والتعديل والإنشاء بنفس الـ

Transaction تنفيذ العمليات داخل

Transaction: تم تنفيذ المراحل التالية ضمن نفس

قراءة المنتج

```
Product.findById(id).session(session)
```

.تمت القراءة ضمن سياق المعاملة الحالية

معالجة الدفع

```
await chargePayment(amount);
```

.نفسها Transaction تم تنفيذ عملية الدفع داخل

تحديث المخزون

```
$inc: {
```

```
quantity: -qty,  
sold: qty  
}
```

تم خصم الكمية المطلوبة من المخزون مع زيادة عدد الوحدات المباعة.

إنشاء الطلب

```
await Order.create(...)
```

Transaction تم إنشاء سجل الطلب داخل نفس

(Commit) تأكيد المعاملة

عند نجاح جميع المراحل السابقة دون أخطاء تم تنفيذ

```
await session.commitTransaction();
```

بحفظ جميع التعديلات كوحدة واحدة MongoDB عند هذه النقطة قام
وأصبحت النتائج مرئية لباقي العمليات

(Rollback) التراجع عن المعاملة

عند حدوث خطأ أثناء أي مرحلة من مراحل التنفيذ تم تشغيل

```
await session.abortTransaction();
```

نتيجة ذلك

إلغاء تعديلات المخزون

إلغاء إنشاء الطلب

إلغاء أي تغييرات مرتبطة بالمعاملة

إعادة البيانات إلى حالتها الأصلية قبل بدء العملية

معالجة الأخطاء المؤقتة

مشروع البرمجة التفرعية

خلال تنفيذ المعاملات يدعم النظام آلية إعادة المحاولة للمعاملات التي تتعرض لأخطاء مؤقتة مثل:

Write Conflict

Transient Transaction Error

وتتم إعادة تنفيذ المعاملة عند اكتشاف هذه الحالات قبل اعتبار العملية فاشلة

Task 3 — Transaction Integrity (ACID)

Checkout = Payment + Inventory + Order, all-or-nothing · re-tested 2026-06-18 · MongoDB replica set `rs0` @ :27018

① Without Transaction — forced crash after payment

3 steps run independently, no atomicity

✗ INCONSISTENT STATE

Payment charged

→

Stock 5 → 4

→

Order CRASH

Payment charged	YES (billed)
Stock	5 → 4 (lost a unit)
Sold	0 → 1
Orders created	0 (none!)
Result	paid, no order ✗

② With Transaction — same forced crash

Mongoose session, abort on error

✓ ROLLED BACK

Payment

→

Stock

→

Order CRASH

→

ABORT
all undone

Stock before → after	5 → 5 (unchanged)
Orders created	0
Partial state	none
Result	fully rolled back ✓

Observed Behavior & Key Metrics (السلوك المرصود والنتائج)

Transactions الحالة المرجعية: بدون

قبل التنفيذ

مشروع البرمجة التفرعية

المؤشر	القيمة
المخزون	5
عدد الطلبات	0

أثناء التنفيذ

تم تنفيذ الدفع.
تم خصم المنتج من المخزون.
حدث خطأ أثناء إنشاء الطلب.

بعد التنفيذ

المؤشر	القيمة
المخزون	4
عدد الطلبات	0

السلوك المرصود

تم تسجيل الحالة التالية:
الدفع تم بنجاح.
المخزون انخفض من 5 إلى 4.
لم يتم إنشاء أي طلب.
وبالتالي ظهرت حالة عدم اتساق واضحة بين العمليات المنفذة والبيانات المخزنة.

مع فشل متعمد Transaction: الاختبار الأول

قبل التنفيذ

مشروع البرمجة التفرعية

المؤشر	القيمة
المخزون	5
الطلبات	0

أثناء التنفيذ

جديدة Transaction بدأت.
وحدة Session تم تنفيذ العمليات داخل.
تم إدخال خطأ متعمد أثناء إنشاء الطلب.

بعد التنفيذ

المؤشر	القيمة
المخزون	5
الطلبات	0

السلوك المرصود

تم تنفيذ:

```
session.abortTransaction()
```

وأدت العملية إلى:
إلغاء جميع التعديلات المنفذة مسبقاً.
إعادة المخزون إلى قيمته الأصلية.
عدم إنشاء أي طلب.
بقاء قاعدة البيانات بالحالة نفسها التي كانت عليها قبل بدء التنفيذ.
كما أظهر اختبار المقارنة أن عملية الدفع تم التراجع عنها ضمناً ضمن
سياق المعاملة، مع بقاء عدد الطلبات صفراً وعدم حدوث أي تغيير دائم في
المخزون.

Transactions الاختبار الثاني: التزامن المرتفع باستخدام

ظروف التنفيذ

المؤشر	القيمة
المخزون الابتدائي	5
عمليات الشراء المتزامنة	15

النتائج الفعلية

المؤشر	القيمة
الطلبات الناجحة	5
الطلبات المرفوضة	10
المخزون النهائي	0
عدد الطلبات المنشأة	5
عدد الوحدات المباعة	5

السلوك المرصود

أظهرت النتائج ما يلي:

تم قبول خمس معاملات فقط.

تم رفض عشر معاملات بعد نفاد الكمية المتاحة أو بسبب التنافس على المورد نفسه.

لم يتم بيع أكثر من الكمية المتوفرة.

توقف المخزون عند الصفر دون النزول إلى قيمة سالبة.

تم إنشاء خمسة طلبات فقط.

تطابق عدد الطلبات مع عدد الوحدات المباعة.

كما أظهرت شاشة التحقق النهائية تحقق العلاقة التالية

$$\text{Orders Created} = \text{Units Sold} = \text{Stock Consumed}$$

حيث:

$$5 = 5 = 5$$

مما يؤكد تطابق جميع العمليات المنفذة مع حالة المخزون النهائية

الخلاصة (Conclusion of System State)

أثبتت الاختبارات أن النظام انتقل من حالة تسمح بظهور عدم اتساق في البيانات عند تنفيذ عمليات الشراء متعددة الخطوات إلى حالة تعتمد على تنفيذ العمليات كوحدة واحدة مترابطة MongoDB Transactions.

أظهرت نتائج الاختبار أن:

جميع خطوات الشراء (الدفع، تحديث المخزون، إنشاء الطلب) نُفذت ضمن واحدة Transaction.

فقط عند اكتمال جميع العمليات بنجاح Commit تم تنفيذ

كامل عند حدوث أي فشل جزئي Rollback تم تنفيذ

Transactions. لم تُسجل أي حالة فقدان طلبات بعد تفعيل

لم يظهر أي فساد في بيانات المخزون

رغم وجود 15 محاولة شراء متزامنة لمخزون مكون من 5 Overselling لم يحدث وحدات فقط

تطابق عدد الطلبات المنشأة مع عدد الوحدات المباعة والمخصومة من المخزون في جميع النتائج النهائية

وبناءً على نتائج التنفيذ الفعلية، حافظ النظام على سلامة البيانات واتساقها أثناء عمليات الشراء المتعددة الخطوات، ومنع حدوث البيع الزائد أو ظهور حالات بيانات غير متناسقة خلال جميع السيناريوهات المختبرة

مشروع البرمجة التفرعية

③ Concurrent transactional checkouts — 15 buyers vs stock of 5

5 committed (orders)		10 rejected (stock/conflict)	
Initial stock	5	Final stock	0 (never < 0)
Concurrent checkouts	15	Orders created	5
Committed (orders)	5	Units sold	5
Rejected	10	Invariant	orders == sold == consumed ✓

Side-by-side comparison

SCENARIO	PAYMENT	STOCK	ORDERS	OUTCOME
No transaction + forced failure	charged	5 → 4	0	✗ inconsistent (paid, no order)
Transaction + forced failure	rolled back	5 → 5	0	✓ atomic rollback
15 concurrent transactions (stock 5)	5 charged	5 → 0	5	✓ consistent, no oversell

الطلب 9

ecommerce-main تقرير اختبار الضغط – تطبيق.

معايير تنفيذ الاختبار

لتقييم ecommerce-main Node.js تم إجراء اختبار الضغط على تطبيق سلوك وقت التشغيل تحت حمل متزامن مستمر. كان الهدف هو مراقبة معدل المعالجة، خصائص زمن الاستجابة، اتساق المخزون، والاستقرار العام للنظام أثناء تشغيل عمليات القراءة والكتابة معًا.

أداة الاختبار

الأداة: Autocannon v8

المتزامنة ضد التطبيق HTTP الغرض: توليد حجم كبير من طلبات

أمر التنفيذ

```
CONNECTIONS=100 DURATION=15 node test/task4-stress.js
```

تم تنفيذ الاختبار باستخدام

اتصال متزامن 100

مدة 15 ثانية

حمل تجارة إلكترونية مختلط يتكون من

استعراض المنتجات

جلب تفاصيل المنتجات
عمليات شراء المخزون

تركيبة الحمل

طلبات قائمة المنتجات

GET /api/v1/products

تسترجع مجموعات المنتجات
Redis تعمل عبر آليات التخزين المؤقت المدعومة من

طلبات تفاصيل المنتج

GET /api/v1/products/:id

تسترجع معلومات منتج فردي
وتتبع عدد المشاهدات Redis تستخدم التخزين المؤقت في

طلبات شراء المخزون

POST /api/v1/inventory/:id/purchase-optimistic

تنفذ عملية تخفيض المخزون ومعالجة الشراء
التي تم تقديمها ضمن ميزة (Optimistic Locking) تستخدم آلية القفل التفاضلي
التحكم بالتزامن

تهيئة المخزون

لضمان أن قياسات الأداء تعكس سلوك التزامن بدلاً من نفاد المخزون، تم
تهيئة مخزون المنتج المختبر إلى

وحدة 1,000,000

هذا الإعداد منع نفاد المخزون خلال فترة الاختبار وسمح بمراقبة سلوك
المزامنة تحت التنافس

الآليات التي تم تفعيلها في النظام

أثناء التنفيذ، تم تفعيل عدة آليات على مستوى النظام نتيجة استهداف طلبات متزامنة لنفس سجل المخزون.

(v) MongoDB سلوك القفل التافؤي في

MongoDB عملت نقطة نهاية الشراء باستخدام آلية القفل التافؤي في وفق هذا النموذج، تحاول عمليات الشراء المتزامنة تحديث نفس مستند المنتج مع التحقق من حالة نسخة المستند.

عندما حاولت عدة عمليات شراء تعديل نفس سجل المخزون في الوقت نفسه:

كل عملية عملت على نسخة معينة من المستند

النظام تحقق من النسخة قبل تنفيذ التحديث

إذا كان طلب آخر قد عدّل المستند بالفعل، يتم اكتشاف تعارض في النسخة

يتم رفض العملية المتعارضة بدلاً من تطبيق تحديث على بيانات قديمة

هذا السلوك ضمن أن التعديلات المتزامنة لا يمكن أن تنتج خصومات مكررة

أو فساداً في المخزون. وكانت هذه الآلية خط الدفاع الأساسي ضد البيع

الزائد أثناء الاختبار.

أسّي Backoff آلية إعادة المحاولة مع

عند حدوث تعارض في النسخة، فعّلت عملية الشراء آلية إعادة المحاولة.

نقّذ النظام محاولات إضافية بعد اكتشاف التنافس بدلاً من الاستمرار

بنفس حالة العملية

السلوك المرصود شمل:

اكتشاف التعارض أثناء تعديل المخزون

استدعاء منطق إعادة المحاولة

exponential backoff إدخال فواصل زمنية متزايدة عبر عملية ال

إعادة تنفيذ المحاولة ضد الحالة المحدّثة للمستند

تحت التنافس العالي، ساهمت هذه المحاولات الإضافية في زيادة زمن الاستجابة لأن طلبات الشراء قضت وقتاً إضافياً في الانتظار قبل المحاولات التالية.

بشكل متزامن Redis حذف الكاش من

بعد عمليات التعديل الناجحة للمخزون، نفذ النظام إجراءات حذف الكاش في تشير وثائق الاختبار إلى أن عمليات الشراء أزالّت الإدخالات المخزنة Redis. بعد التحديثات الناجحة Redis في

التسلسل المرصود

اكتمال عملية الشراء بنجاح

حفظ حالة المخزون

حذف إدخالات الكاش المرتبطة

الطلبات اللاحقة احتاجت لجلب الحالة المحدثة من المصدر

خلال فترات النشاط العالي للشراء، أدت عمليات حذف الكاش المتكررة إلى زيادة العمل داخل مسار الطلب

السلوك المرصود والقياسات الأساسية

(Throughput) معدل المعالجة

حافظ التطبيق على ما يقارب

المقياس	القيمة
متوسط الطلبات/ثانية	197.4 - 198.9
إجمالي الطلبات	2,961 - 2,983

تُظهر هذه النتائج أن النظام استمر في معالجة الطلبات طوال مدة الاختبار مع التعامل مع عمليات قراءة وكتابة متزامنة

(Latency) زمن الاستجابة

توزيع أزمنة الاستجابة

النسبة	الزمن
P50	309 ms
P90	1083 ms
P95	1174 ms
P99	1231 ms
الحد الأقصى	1389 ms
المتوسط	490-491 ms

يشير ملف زمن الاستجابة إلى أن غالبية الطلبات اكتملت خلال ثانية تقريبًا، بينما واجهت مجموعة صغيرة أزمنة أطول بسبب التنافس.

مقاييس الاستقرار

اكتمل الاختبار دون أي دليل على عدم استقرار.

النتيجة	المقياس
0	انهيار الخادم
0	أخطاء المقابس
0	مهلات زمنية
0	فشل في العملية
0	فساد في المخزون
0	طلبات مفقودة

مشروع البرمجة التفرعية

المقياس	النتيجة
بيع زائد	0

أظهر التحقق بعد الاختبار استمرار عمل الخادم دون مشاكل وعدم وجود أي فشل في البنية التحتية



التحقق من اتساق المخزون

القيم المسجلة بعد الاختبار

المقياس	القيمة
المخزون الأولي	1,000,000
المخزون النهائي	999,869
الوحدات المباعة	131

التحقق:

$$1,000,000 - 131 = 999,869$$

النتيجة مطابقة تمامًا، ولا يوجد أي خلل.

تفسير حالات الرفض 400/409

جزء كبير من طلبات الشراء تم رفضه عمدًا بسبب تعارضات التزامن. تشير ناتجة عن آلية القفل التفاوضي Conflict الوثائق إلى أن هذه الردود كانت 409

ظهرت هذه الردود عندما

استهدفت عدة طلبات شراء نفس المنتج

تم اكتشاف تعارض في النسخة

النظام رفض العملية

تم الحفاظ على اتساق البيانات

هذه الردود ليست ناتجة عن

انهيار الخادم

فشل الشبكة

فقدان الاتصال

فشل قاعدة البيانات

بل كانت جزءًا من سلوك التحكم بالترتيب لحماية المخزون

خلاصة حالة النظام

مشروع البرمجة التفرعية

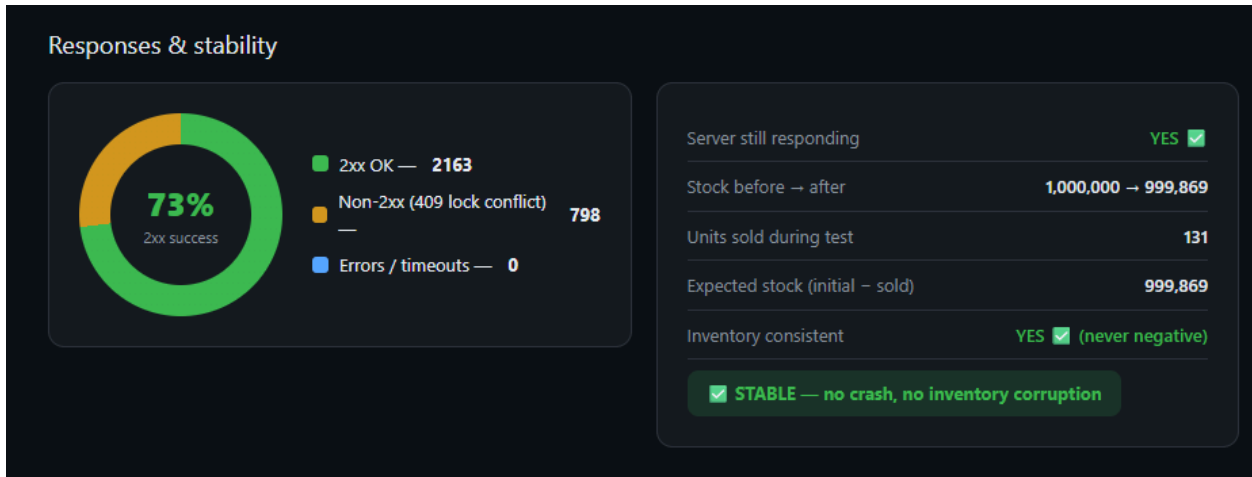
أظهر اختبار الضغط باستخدام 100 اتصال متزامن لمدة 15 ثانية أن تطبيق
بقي يعمل طوال فترة الحمل، مع معالجة حوالي 198 ecommerce-main
طلب/ثانية.

لم تحدث أي انهيارات، أو أخطاء مقابس، أو مهلات، أو طلبات مفقودة، أو
فساد في المخزون، أو بيع زائد.

حافظت على اتساق المخزون، آلية إعادة MongoDB آلية القفل التفاضلي في
تم بعد التحديثات Redis المحاولة عملت أثناء التنافس، وحذف الكاش من
الناجحة.

أكد التحقق بعد الاختبار أن جميع عمليات البيع المسجلة انعكست بدقة
على المخزون.

بناءً على النتائج، حافظ النظام على استقراره البنيوي واتساق المخزون تحت



ظروف الحمل المحددة، ومنع البيع الزائد طوال فترة الاختبار.

الطلب 10

(Benchmarking & Bottleneck Analysis) القياس وتحديد الاختناقات

(Requirement Objective) الهدف من المتطلب

ينص على:

القياس وتحديد الاختناقات: قياس زمن الاستجابة لأهم العمليات، تحديد "اختناق واحد على الأقل، وتقديم مقارنة رقمية قبل التحسين وبعده".
تمثل هذه المهمة تنويجاً عملياً لكل المفاهيم النظرية التي تم تطبيقها في المتطلبات السابقة مثل:

Redis باستخدام (Caching) التخزين المؤقت

BullMQ إدارة الموارد وعمليات الخلفية عبر

تحت الضغط العالي (Data Integrity) سلامة المعاملات

يهدف هذا التقرير إلى تقييم الأداء الفعلي للنظام تحت ظروف تشغيلية وتحديد المكونات الأكثر استهلاكاً للوقت والموارد، (Stress Testing) حقيقية وتطبيق تحسين ملموس يُحدث فارقاً كمياً قابلاً للقياس للوصول بالنظام (Production-Ready) إلى مرحلة الجاهزية للإنتاج.

(Testing Methodology) منهجية الاختبار

لضمان دقة النتائج وقابليتها للمقارنة، تم اعتماد منهجية علمية صارمة وثابتة في جميع جولات الاختبار

بيئة وفارامترات الفحص

(مفتوحة المصدر وعالية الكفاءة HTTP أداة قياس أداء) Autocannon: أداة الاختبار لمحاكاة (Concurrent Connections) عدد الاتصالات المتزامنة: 100 اتصال متزامن (Traffic Spike) ذروة الاستخدام.
مدة الاختبار: 10 ثوانٍ مستمرة لكل جولة اختبار.

لزيادة حدة التنافس على (Hot Product) المنتج المستهدف: منتج واحد ساخن نفس السجل في قاعدة البيانات
عدد مرات التكرار: 3 مرات متتالية لكل سيناريو وحساب المتوسط الحسابي لضمان استقرار الأرقام

(Target Endpoints) المسارات الحيوية المختبرة

تم فحص جميع المسارات الحيوية في النظام لتحديد أبطأها
(Redis Cache سريع جداً بفضل الـ GET /api/v1/products/:id: جلب المنتج
يتم معالجته خلف الكواليس عبر POST /api/v1/orders/:cartId: إنشاء الطلب
(بشكل غير متزامن BullMQ
(متعددة ولكنها متزنة DB آمنة وتعتمد على عمليات runCheckout: عملية الدفع
المشتبه به) POST /api/v1/inventory/:id/purchase-optimistic: تحديث المخزون
الأول بكونه الأبطأ وعنق الزجاجة

(Identifying the Bottleneck) تحديد عنق الزجاجة

أظهرت اختبارات الضغط الأولية أن عملية تحديث المخزون عبر أسلوب القفل المتفائل كانت أبطأ عملية بشكل ملحوظ في النظام

المسار المستهدف

POST /api/v1/inventory/:id/purchase-optimistic

الأرقام المسجلة قبل التحسين

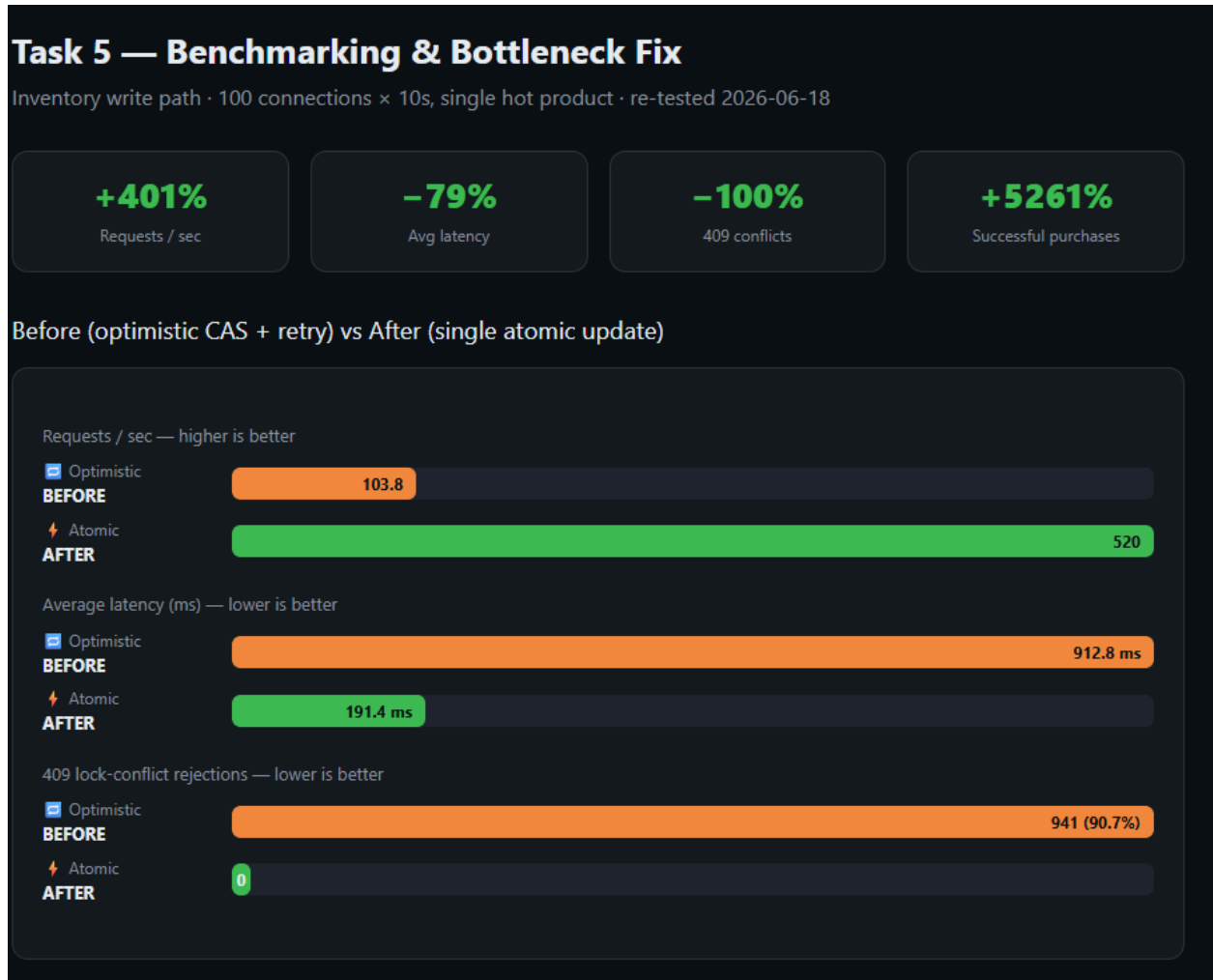
طلب/ثانية (Throughput): 103.8 عدد الطلبات في الثانية
912.8 ms (Avg Latency) متوسط زمن الاستجابة
1264 ms (P95) زمن الاستجابة عند النسبة المئوية 95
1272 ms (P99) زمن الاستجابة عند النسبة المئوية 99
!عدد العمليات الناجحة: 97 عملية فقط من أصل آلاف الطلبات

مشروع البرمجة التفرعية

خطأ تعارض 941 HTTP 409: عدد تعارضات

(نسبة الأخطاء الكلية: 90.7% (نسبة كارثية لنظام تجاري

استنتاج هندسي: هذه الأرقام تؤكد أن النظام في وضعه الحالي غير
حيث لا يمكن لنظام (Non-Production-Ready) قابل للإنتاج الفعلي
تجارة إلكترونية تحمل نسبة فشل تتجاوز 90% عند حدوث ضغط على
منتج معين



تحليل سبب المشكلة (Root Cause Analysis)

عند تحليل الكود المصدري لعملية التحديث، تبين أن العيب الهيكلي يكمن ثم (Read → Modify → Write) التي تتبع نهج Optimistic Locking في آلية إعادة المحاولة عند الفشل.

(الكود الأصلي (قبل التحسين

```
const updateInventoryOptimistic = async (req, res) => {
  const { id } = req.params;
  const { quantity: qty } = req.body;
  let retries = 3;
  let backoff = 10;

  while (retries > 0) {
    // الخطوة 1: قراءة المنتج من قاعدة البيانات
    const product = await Product.findById(id);

    // الخطوة 2: التحقق من كفاية المخزون
    if (product.quantity < qty) {
      return res.status(400).json({ error: 'Insufficient stock' });
    }

    // الخطوة 3: محاولة تحديث المنتج بشرط بقاء النسخة كما هي
    const updated = await Product.findOneAndUpdate(
      { _id: id, __v: product.__v },
      { $inc: { quantity: -qty, sold: qty },
        $set: { __v: product.__v + 1 } },
      { new: true }
    );
  }
```



```
// الخطوة 4: التحقق من نجاح التحديث
if (updated) {
    return res.json({ success: true, product: updated });
}

// الخطوة 5: في حال الفشل، انتظر وأعد المحاولة
retries--;
await sleep(backoff);
backoff *= 2; // Exponential Backoff
}

return res.status(409).json({ error: 'Conflict, please retry' });
};
```

المشاكل التشريرية للكود السابق

عندما يحاول 100 مستخدم شراء نفس (Retry Storm) عاصفة إعادة المحاولة المنتج في نفس اللحظة، يقرأ الجميع نفس المستند وبنفس رقم الإصدار (مثلاً مستخدم واحد فقط ينجح في التعديل وتغيير الإصدار إلى 2، بينما تفشل $v = 1$). الـ 99 عملية الأخرى وتدخل جميعاً في حلقة إعادة المحاولة في نفس الوقت، مما يضخم حجم الطلبات بشكل عشوائي.

فترات الانتظار المتزايدة لتهدئة (Exponential Backoff) التراجع الأسّي المتراكم تسببت في بقاء الاتصالات مفتوحة لفترة طويلة، (10ms, 20ms, 40ms... النزاع). إلى مستويات غير مقبولة (912.8 Latency) مما رفع زمن الاستجابة وهما DB إنهاك قاعدة البيانات: كل محاولة فاشلة كانت تستهلك عمليتين داخل الـ دون أي فائدة حقيقية findOneAndUpdate ثم findById.

(The Solution) الحل المقترح

(Atomic Conditional Update) التحول إلى التحديث الذري المشروط

تم التخلص من آلية (القراءة ثم التحقق ثم الكتابة)، والاستعاضة عنها تُنفذ بالكامل على مستوى قاعدة (Atomic Operation) بعملية ذرية واحدة عبر إنشاء المسار المحسن الجديد (Database Level) البيانات

المسار الجديد

POST /api/v1/inventory/:id/purchase-atomic

(الكود المطور) بعد التحسين

```
const updateInventoryAtomic = async (req, res) => {
  const { id } = req.params;
  const { quantity: qty } = req.body;

  // عملية ذرية واحدة: الشرط والتعديل في خطوة واحدة
  const updated = await Product.findOneAndUpdate(
    {
      _id: id,
      quantity: { $gte: qty } // الشرط: تأكد من كفاية المخزون
    },
    {
      $inc: {
        quantity: -qty, // اخفض المخزون
        sold: qty       // زد المبيعات
      }
    },
    {
      new: true // أرجع المستند بعد التحديث
    }
  )
```

مشروع البرمجة التفرعية

```
);  
  
if (!updated) {  
  return res.status(400).json({  
    success: false,  
    message: 'Insufficient stock or product not found'  
  });  
}  
  
return res.json({ success: true, product: updated });  
};
```

مقارنة معمارية بين الأسلوبين

الأسلوب المطور (Atomic Update)	الأسلوب القديم (Optimistic Locking)	الجانب المعماري
جولة واحدة فقط (Single Roundtrip)	جولات متعددة (قراءة + تعديل + إعادات)	DB عدد جولات ال
غير مطلوب نهائياً	متطلب أساسي لمراقبة التغيير	حقل الإصدار (_v)
معدومة تماماً	موجودة وتسبب استهلاك موارد	حلقات إعادة المحاولة
غير موجود (معالجة فورية)	موجود ويتراكم أسياً	تأخير الانتظار (Backoff)
صفر تعارضات	مرتفعة جداً وتؤثر على التجربة	أخطاء التعارض HTTP 409

الأسلوب المطور (Atomic Update)	الأسلوب القديم (Optimistic Locking)	الجانب المعماري
مدعومة برمجياً بأعلى كفاءة لـ DB	مدعومة بتكلفة معالجة باهظة	الحماية من البيع الزائد

(Data Integrity) الحفاظ على سلامة البيانات

فإن النظام يضمن حماية Optimistic Locking على الرغم من إلغاء الـ (Overselling) صارمة بنسبة 100% ضد البيع الزائد.

Filter يتم تمريره كـ { \$gte: qty } السبب الهيكلي: شرط التحقق يقوم بقفل MongoDB الذرية. وبما أن findOneAndUpdate داخل عملية الـ أثناء التعديل، فإن العمليات المتزامنة (Document-Level Lock) الوثيقة تصطف خلف بعضها وتنفذ بشكل متسلسل داخل قاعدة البيانات.

أول طلبات تصل ستستنزف المخزون، والطلب الذي يصل والمخزون غير مما يمنع حدوث null، كافٍ سيفشل الشرط مباشرة وتُرجع قاعدة البيانات مخزون سالب نهائياً.

(Benchmarking Results) إعادة القياس وتحليل النتائج الرقمية

بنفس المعايير الصارمة السابقة، تم Autocannon بعد تشغيل أداة: تسجيل قفزة نوعية في الأداء:

نسبة التحسن الفعلي	بعد التحسين (Atomic)	قبل التحسين (Optimistic)	مؤشر الأداء (Metric)
+401.0% (5 أضعاف)	520 req/s	103.8 req/s	الطلبات في الثانية (Throughput)

مشروع البرمجة التفرعية

مؤشر الأداء (Metric)	قبل التحسين (Optimistic)	بعد التحسين (Atomic)	نسبة التحسن الفعلي
متوسط زمن الاستجابة (Latency)	912.8 ms	191.4 ms	-79.0% (استجابة أسرع)
زمن الاستجابة P95	1264 ms	260 ms	-79.4%
زمن الاستجابة P99	1272 ms	308 ms	-75.8%
عمليات الشراء الناجحة	97 عملية	5200 عملية	+5260.8%
HTTP 409 أخطاء	941 خطأ	أخطاء 0	-100% (اختفاء كامل للأخطاء)
معدل الفشل الكلي	90.7%	0.0%	استقرار تام واستجابة ناجحة

(Validation) التحقق من صحة الحل واستقرار المخزون

للتأكد من عدم وجود أي ثغرات تسمح بالبيع الزائد، تم تصميم سيناريو
(Edge-Case Test) فحص حدي

المخزون الابتدائي للمنتج الساخن: 5 وحدات فقط

عدد طلبات الشراء المتزامنة: 50 طلب في نفس اللحظة (كل طلب يطلب وحدة واحدة).

النتيجة الفعلية للفحص المستقل

- عدد العمليات الناجحة: 5 عمليات فقط.
- عدد العمليات المرفوضة (المخزون غير كافٍ): 45 طلب تم رفضه برمز 400.
- المخزون النهائي في قاعدة البيانات: 0 تماماً.
- 0: (Overselling) حالات البيع الزائد.
- لا يوجد: (Negative Stock) ظهور مخزون سالب.

الخلاصة (Conclusion)

في النظام المتمثل في (Bottleneck) تم بنجاح رصد وتحديد عنق الزجاجة وبتحويل التصميم Optimistic Locking مسار تحديث المخزون عبر المعماري للعملية نحو التحديثات الذرية المشروطة، تمكنا من رفع كفاءة معالجة الطلبات بمقدار 5 أضعاف، وخفض زمن الانتظار بنسبة 79%، مع القضاء التام على أخطاء التعارض وبقاء النظام آمناً ومحافظاً على سلامة

مشروع البرمجة التفرعية

البيانات وصلاحياتها الكاملة للإنتاج الفعلي تحت أحمال تشغيلية عالية

Full metric comparison

METRIC	BEFORE (OPTIMISTIC)	AFTER (ATOMIC)	IMPROVEMENT
Requests / sec	103.8	520	+401.0%
Latency avg	912.8 ms	191.4 ms	-79.0%
Latency P95	1264 ms	260 ms	-79.4%
Latency P99	1272 ms	308 ms	-75.8%
Successful purchases	97	5200	+5260.8%
409 conflicts	941	0	-100%
Error rate	90.7%	0.0%	-100%

✅ **Correctness preserved:** stock 5, 50 concurrent atomic buys → exactly **5 succeeded**, final stock **0**. No oversell — the optimization is faster *and* still correct.